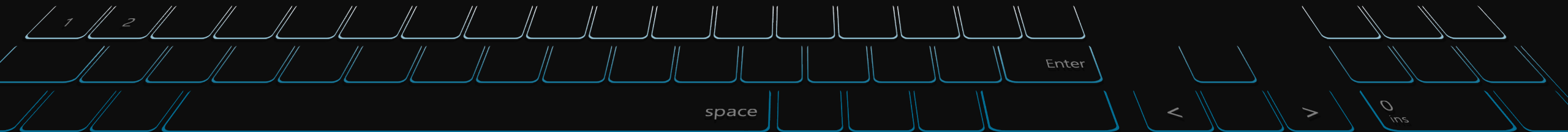


06 데이터 정제 및 준비 (조인, 병합, 변형)





강의 내용

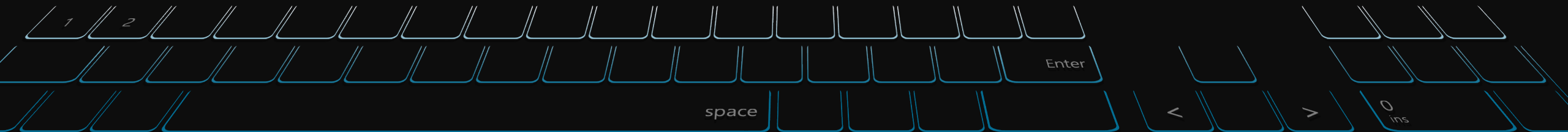
Contents of Lecture

기간	내용	과제
01주차	오리엔테이션 Python 언어의 기본, lpython, 주피터 노트북	-
02주차	내장 자료구조, 함수, 파일	실습 과제
03주차	Numpy 기본 : 배열과 벡터 연산	실습 과제
04주차	Pandas 시작하기	실습 과제
05주차	데이터 로딩과 저장, 파일 형식	실습 과제
06주차	데이터 정제 및 준비(조인, 병합, 변형)	실습 과제
07주차	그래프와 시각화	기말 프로젝트 상세 공지
08주차	중간고사 - 오프라인 오픈북으로 진행	실습 과제

기간	내용	과제
09주차	데이터 집계와 그룹 연산	실습 과제
10주차	시계열	실습 과제
11주차	고급 pandas	실습 과제
12주차	파이썬 모델링 라이브러리	실습 과제
13주차	데이터 분석 예제 / 고급 Numpy	실습 과제
14주차	Kaggle / Dacon 데이터 분석 사례 Case Study	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!

1부: 데이터 정제 및 준비

- 1) 누락된 데이터 처리하기
- 2) 데이터 변형
- 3) 문자열 다루기





데이터 정제 및 준비

Reading and Writing Data in Text Format

- 데이터 분석 및 모델링을 수행시,
데이터 로드, 정리, 변환 및 재정렬과 같은 데이터 준비에 상당한 시간 소요
- 이러한 작업은 종종 분석가의 시간 중 80% 이상을 차지



[그림 1: 공공 빅데이터 분석과제 추진 프로세스]

□ 추진 절차

추진체계 구성	• 추진 체계 구성
빅데이터 분석과제 발굴	• 분석 주제 도출 • 주제 중복성 검토 • 기 분석모델 활용 검토
데이터 확보 방안 검토	• 필요 데이터 정의 • 기관 내/외부 데이터 확보방안 검토 • 행정정보, 공공 데이터 활용방안 검토
분석과제 선정	• 분석 추진 가능성 검토 • 분석과제 확정
분석 방안 수립	• 분석 목표 정의 • 분석 방안 제시
분석 결과 활용계획 수립	• 분석 결과 활용방안 도출 • 분석 결과 홍보방안 고려
과제 추진계획 수립	• 과제 목표 및 기본 요건 정의 • 과제 추진방법 선정 • 분석 비용 및 기간 산정

[행정안전부 링크] [2016.12.28 공공분야 빅데이터 업무적용 및 대가산정 가이드](#)



7.1 Handling Missing Data

Missing data occurs commonly in many data analysis applications. One of the goals of pandas is to make working with missing data as painless as possible. For example, all of the descriptive statistics on pandas objects exclude missing data by default.

The way that missing data is represented in pandas objects is somewhat imperfect, but it is functional for a lot of users. For numeric data, pandas uses the floating-point value `NaN` (Not a Number) to represent missing data. We call this a *sentinel value* that can be easily detected:

```
In [10]: string_data = pd.Series(['aardvark', 'artichoke', np.nan, 'avocado'])
```

```
In [11]: string_data
```

```
Out[11]:
0    aardvark
1    artichoke
2         NaN
3     avocado
dtype: object
```

```
In [12]: string_data.isnull()
```

```
Out[12]:
0    False
1    False
2     True
3    False
dtype: bool
```

In pandas, we've adopted a convention used in the R programming language by referring to missing data as NA, which stands for *not available*. In statistics applications, NA data may either be data that does not exist or that exists but was not observed (through problems with data collection, for example). When cleaning up data for analysis, it is often important to do analysis on the missing data itself to identify data collection problems or potential biases in the data caused by missing data.

The built-in Python None value is also treated as NA in object arrays:

```
In [13]: string_data[0] = None
```

```
In [14]: string_data.isnull()
```

```
Out[14]:
0     True
1    False
2     True
3    False
dtype: bool
```

There is work ongoing in the pandas project to improve the internal details of how missing data is handled, but the user API functions, like `pandas.isnull`, abstract away many of the annoying details. See [Table 7-1](#) for a list of some functions related to missing data handling.

Table 7-1. NA handling methods

Argument	Description
<code>dropna</code>	Filter axis labels based on whether values for each label have missing data, with varying thresholds for how much missing data to tolerate.
<code>fillna</code>	Fill in missing data with some value or using an interpolation method such as 'ffill' or 'bfill'.
<code>isnull</code>	Return boolean values indicating which values are missing/NA.
<code>notnull</code>	Negation of <code>isnull</code> .



누락된 데이터 처리하기

Handling Missing Data

Filtering Out Missing Data

There are a few ways to filter out missing data. While you always have the option to do it by hand using `pandas.isnull` and boolean indexing, the `dropna` can be helpful. On a Series, it returns the Series with only the non-null data and index values:

```
In [15]: from numpy import nan as NA
```

```
In [16]: data = pd.Series([1, NA, 3.5, NA, 7])
```

```
In [17]: data.dropna()
```

```
Out[17]:
```

```
0    1.0
```

```
2    3.5
```

```
4    7.0
```

```
dtype: float64
```

This is equivalent to:

```
In [18]: data[data.notnull()]
```

```
Out[18]:
```

```
0    1.0
```

```
2    3.5
```

```
4    7.0
```

```
dtype: float64
```

With DataFrame objects, things are a bit more complex. You may want to drop rows or columns that are all NA or only those containing any NAs. `dropna` by default drops any row containing a missing value:

```
In [19]: data = pd.DataFrame([[1., 6.5, 3.], [1., NA, NA],  
.....:                      [NA, NA, NA], [NA, 6.5, 3.]])
```

```
In [20]: cleaned = data.dropna()
```

```
In [21]: data
```

```
Out[21]:
```

```
      0      1      2
```

```
0  1.0  6.5  3.0
```

```
1  1.0  NaN  NaN
```

```
2  NaN  NaN  NaN
```

```
3  NaN  6.5  3.0
```

```
In [22]: cleaned
```

```
Out[22]:
```

```
      0      1      2
```

```
0  1.0  6.5  3.0
```



누락된 데이터 처리하기

Handling Missing Data

Passing how='all' will only drop rows that are all NA:

```
In [23]: data.dropna(how='all')
Out[23]:
```

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
3	NaN	6.5	3.0

To drop columns in the same way, pass axis=1:

```
In [24]: data[4] = NA
```

```
In [25]: data
Out[25]:
```

	0	1	2	4
0	1.0	6.5	3.0	NaN
1	1.0	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	6.5	3.0	NaN

```
In [26]: data.dropna(axis=1, how='all')
Out[26]:
```

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
2	NaN	NaN	NaN
3	NaN	6.5	3.0

A related way to filter out DataFrame rows tends to concern time series data. Suppose you want to keep only rows containing a certain number of observations. You can indicate this with the thresh argument:

```
In [27]: df = pd.DataFrame(np.random.randn(7, 3))
```

```
In [28]: df.iloc[:4, 1] = NA
```

```
In [29]: df.iloc[:2, 2] = NA
```

```
In [30]: df
Out[30]:
```

	0	1	2
0	-0.204708	NaN	NaN
1	-0.555730	NaN	NaN
2	0.092908	NaN	0.769023
3	1.246435	NaN	-1.296221
4	0.274992	0.228913	1.352917
5	0.886429	-2.001637	-0.371843
6	1.669025	-0.438570	-0.539741

```
In [31]: df.dropna()
Out[31]:
```

	0	1	2
4	0.274992	0.228913	1.352917
5	0.886429	-2.001637	-0.371843
6	1.669025	-0.438570	-0.539741

```
In [32]: df.dropna(thresh=2)
Out[32]:
```

	0	1	2
2	0.092908	NaN	0.769023
3	1.246435	NaN	-1.296221
4	0.274992	0.228913	1.352917
5	0.886429	-2.001637	-0.371843
6	1.669025	-0.438570	-0.539741



누락된 데이터 처리하기

Handling Missing Data

Filling In Missing Data

Rather than filtering out missing data (and potentially discarding other data along with it), you may want to fill in the “holes” in any number of ways. For most purposes, the `fillna` method is the workhorse function to use. Calling `fillna` with a constant replaces missing values with that value:

```
In [33]: df.fillna(0)
Out[33]:
```

	0	1	2
0	-0.204708	0.000000	0.000000
1	-0.555730	0.000000	0.000000
2	0.092908	0.000000	0.769023
3	1.246435	0.000000	-1.296221
4	0.274992	0.228913	1.352917
5	0.886429	-2.001637	-0.371843
6	1.669025	-0.438570	-0.539741

Calling `fillna` with a dict, you can use a different fill value for each column:

```
In [34]: df.fillna({1: 0.5, 2: 0})
Out[34]:
```

	0	1	2
0	-0.204708	0.500000	0.000000
1	-0.555730	0.500000	0.000000
2	0.092908	0.500000	0.769023
3	1.246435	0.500000	-1.296221
4	0.274992	0.228913	1.352917
5	0.886429	-2.001637	-0.371843
6	1.669025	-0.438570	-0.539741

`fillna` returns a new object, but you can modify the existing object in-place:

```
In [35]: _ = df.fillna(0, inplace=True)
```

```
In [36]: df
Out[36]:
```

	0	1	2
0	-0.204708	0.000000	0.000000
1	-0.555730	0.000000	0.000000
2	0.092908	0.000000	0.769023
3	1.246435	0.000000	-1.296221
4	0.274992	0.228913	1.352917
5	0.886429	-2.001637	-0.371843
6	1.669025	-0.438570	-0.539741



누락된 데이터 처리하기

Handling Missing Data

The same interpolation methods available for reindexing can be used with `fillna`:

```
In [37]: df = pd.DataFrame(np.random.randn(6, 3))
```

```
In [38]: df.iloc[2:, 1] = NA
```

```
In [39]: df.iloc[4:, 2] = NA
```

```
In [40]: df
```

```
Out[40]:
      0      1      2
0  0.476985  3.248944 -1.021228
1 -0.577087  0.124121  0.302614
2  0.523772      NaN  1.343810
3 -0.713544      NaN -2.370232
4 -1.860761      NaN      NaN
5 -1.265934      NaN      NaN
```

```
In [41]: df.fillna(method='ffill')
```

```
Out[41]:
      0      1      2
0  0.476985  3.248944 -1.021228
1 -0.577087  0.124121  0.302614
2  0.523772  0.124121  1.343810
3 -0.713544  0.124121 -2.370232
4 -1.860761  0.124121 -2.370232
5 -1.265934  0.124121 -2.370232
```

```
In [42]: df.fillna(method='ffill', limit=2)
```

```
Out[42]:
      0      1      2
0  0.476985  3.248944 -1.021228
1 -0.577087  0.124121  0.302614
2  0.523772  0.124121  1.343810
3 -0.713544  0.124121 -2.370232
4 -1.860761      NaN -2.370232
5 -1.265934      NaN -2.370232
```

With `fillna` you can do lots of other things with a little creativity. For example, you might pass the mean or median value of a Series:

```
In [43]: data = pd.Series([1., NA, 3.5, NA, 7])
```

```
In [44]: data.fillna(data.mean())
```

```
Out[44]:
0    1.000000
1    3.833333
2    3.500000
3    3.833333
4    7.000000
dtype: float64
```

See [Table 7-2](#) for a reference on `fillna`.

Table 7-2. `fillna` function arguments

Argument	Description
<code>value</code>	Scalar value or dict-like object to use to fill missing values
<code>method</code>	Interpolation; by default 'ffill' if function called with no other arguments
<code>axis</code>	Axis to fill on; default axis=0
<code>inplace</code>	Modify the calling object without producing a copy
<code>limit</code>	For forward and backward filling, maximum number of consecutive periods to fill



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

Removing Duplicates

Duplicate rows may be found in a DataFrame for any number of reasons. Here is an example:

```
In [45]: data = pd.DataFrame({'k1': ['one', 'two'] * 3 + ['two'],  
.....:                      'k2': [1, 1, 2, 3, 3, 4, 4]})
```

```
In [46]: data
```

```
Out[46]:
```

	k1	k2
0	one	1
1	two	1
2	one	2
3	two	3
4	one	3
5	two	4
6	two	4

The DataFrame method `data.duplicated` returns a boolean Series indicating whether each row is a duplicate (has been observed in a previous row) or not:

```
In [47]: data.duplicated()
```

```
Out[47]:
```

0	False
1	False
2	False
3	False
4	False
5	False
6	True

dtype: bool

Relatedly, `drop_duplicates` returns a DataFrame where the duplicated array is False:

```
In [48]: data.drop_duplicates()
```

```
Out[48]:
```

	k1	k2
0	one	1
1	two	1
2	one	2
3	two	3
4	one	3
5	two	4

Both of these methods by default consider all of the columns; alternatively, you can specify any subset of them to detect duplicates. Suppose we had an additional column of values and wanted to filter duplicates only based on the 'k1' column:

```
In [49]: data['v1'] = range(7)
```

```
In [50]: data.drop_duplicates(['k1'])
```

```
Out[50]:
```

	k1	k2	v1
0	one	1	0
1	two	1	1

`duplicated` and `drop_duplicates` by default keep the first observed value combination. Passing `keep='last'` will return the last one:

```
In [51]: data.drop_duplicates(['k1', 'k2'], keep='last')
```

```
Out[51]:
```

	k1	k2	v1
0	one	1	0
1	two	1	1
2	one	2	2
3	two	3	3
4	one	3	4
6	two	4	6



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

Transforming Data Using a Function or Mapping

For many datasets, you may wish to perform some transformation based on the values in an array, Series, or column in a DataFrame. Consider the following hypothetical data collected about various kinds of meat:

```
In [52]: data = pd.DataFrame({'food': ['bacon', 'pulled pork', 'bacon',
....:                                'Pastrami', 'corned beef', 'Bacon',
....:                                'pastrami', 'honey ham', 'nova lox'],
....:                        'ounces': [4, 3, 12, 6, 7.5, 8, 3, 5, 6]})
```

In [53]: data

```
Out[53]:
```

	food	ounces
0	bacon	4.0
3	Pastrami	6.0
4	corned beef	7.5
5	Bacon	8.0
6	pastrami	3.0
7	honey ham	5.0
8	nova lox	6.0

Suppose you wanted to add a column indicating the type of animal that each food came from. Let's write down a mapping of each distinct meat type to the kind of animal:

```
meat_to_animal = {
    'bacon': 'pig',
    'pulled pork': 'pig',
    'pastrami': 'cow',
    'corned beef': 'cow',
    'honey ham': 'pig',
    'nova lox': 'salmon'
}
```

The map method on a Series accepts a function or dict-like object containing a mapping, but here we have a small problem in that some of the meats are capitalized and others are not. Thus, we need to convert each value to lowercase using the `str.lower` Series method:

```
In [55]: lowercased = data['food'].str.lower()
```

In [56]: lowercased

```
Out[56]:
```

0	bacon
1	pulled pork
2	bacon
3	pastrami
4	corned beef
5	bacon
6	pastrami
7	honey ham
8	nova lox

Name: food, dtype: object



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

```
In [57]: data['animal'] = lowercased.map(meat_to_animal)
```

```
In [58]: data
```

```
Out[58]:
```

	food	ounces	animal
0	bacon	4.0	pig
1	pulled pork	3.0	pig
2	bacon	12.0	pig
3	Pastrami	6.0	cow
4	corned beef	7.5	cow
5	Bacon	8.0	pig
6	pastrami	3.0	cow
7	honey ham	5.0	pig
8	nova lox	6.0	salmon

We could also have passed a function that does all the work:

```
In [59]: data['food'].map(lambda x: meat_to_animal[x.lower()])
```

```
Out[59]:
```

0	pig
1	pig
2	pig
3	cow
4	cow
5	pig
6	cow
7	pig
8	salmon

Name: food, dtype: object

Using map is a convenient way to perform element-wise transformations and other data cleaning-related operations.



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

Replacing Values

Filling in missing data with the `fillna` method is a special case of more general value replacement. As you've already seen, `map` can be used to modify a subset of values in an object but `replace` provides a simpler and more flexible way to do so. Let's consider this Series:

```
In [60]: data = pd.Series([1., -999., 2., -999., -1000., 3.])
```

```
In [61]: data
```

```
Out[61]:  
0      1.0  
1    -999.0  
2      2.0  
3    -999.0  
4   -1000.0  
5      3.0  
dtype: float64
```

The -999 values might be sentinel values for missing data. To replace these with NA values that pandas understands, we can use `replace`, producing a new Series (unless you pass `inplace=True`):

```
In [62]: data.replace(-999, np.nan)
```

```
Out[62]:  
0      1.0  
1      NaN  
2      2.0  
3      NaN  
4   -1000.0  
5      3.0  
dtype: float64
```

If you want to replace multiple values at once, you instead pass a list and then the substitute value:

```
In [63]: data.replace([-999, -1000], np.nan)
```

```
Out[63]:  
0      1.0  
1      NaN  
2      2.0  
3      NaN  
4      NaN  
5      3.0  
dtype: float64
```

To use a different replacement for each value, pass a list of substitutes:

```
In [64]: data.replace([-999, -1000], [np.nan, 0])
```

```
Out[64]:  
0      1.0  
1      NaN  
2      2.0  
3      NaN  
4      0.0  
5      3.0  
dtype: float64
```

The argument passed can also be a dict:

```
In [65]: data.replace({-999: np.nan, -1000: 0})
```

```
Out[65]:  
0      1.0  
1      NaN  
2      2.0  
3      NaN  
4      0.0  
5      3.0  
dtype: float64
```



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

Renaming Axis Indexes

Like values in a Series, axis labels can be similarly transformed by a function or mapping of some form to produce new, differently labeled objects. You can also modify the axes in-place without creating a new data structure. Here's a simple example:

```
In [66]: data = pd.DataFrame(np.arange(12).reshape((3, 4)),
.....:                      index=['Ohio', 'Colorado', 'New York'],
.....:                      columns=['one', 'two', 'three', 'four'])
```

Like a Series, the axis indexes have a map method:

```
In [67]: transform = lambda x: x[:4].upper()

In [68]: data.index.map(transform)
Out[68]: Index(['OHIO', 'COLO', 'NEW'], dtype='object')
```

You can assign to index, modifying the DataFrame in-place:

```
In [69]: data.index = data.index.map(transform)

In [70]: data
Out[70]:
```

	one	two	three	four
OHIO	0	1	2	3
COLO	4	5	6	7
NEW	8	9	10	11

If you want to create a transformed version of a dataset without modifying the original, a useful method is rename:

```
In [71]: data.rename(index=str.title, columns=str.upper)
Out[71]:
```

	ONE	TWO	THREE	FOUR
Ohio	0	1	2	3
Colo	4	5	6	7
New	8	9	10	11

Notably, rename can be used in conjunction with a dict-like object providing new values for a subset of the axis labels:

```
In [72]: data.rename(index={'OHIO': 'INDIANA'},
.....:                columns={'three': 'peekaboo'})
Out[72]:
```

	one	two	peekaboo	four
INDIANA	0	1	2	3
COLO	4	5	6	7
NEW	8	9	10	11

rename saves you from the chore of copying the DataFrame manually and assigning to its index and columns attributes. Should you wish to modify a dataset in-place, pass inplace=True:

```
In [73]: data.rename(index={'OHIO': 'INDIANA'}, inplace=True)

In [74]: data
Out[74]:
```

	one	two	three	four
INDIANA	0	1	2	3
COLO	4	5	6	7
NEW	8	9	10	11



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

Discretization and Binning

Continuous data is often discretized or otherwise separated into “bins” for analysis. Suppose you have data about a group of people in a study, and you want to group them into discrete age buckets:

```
In [75]: ages = [20, 22, 25, 27, 21, 23, 37, 31, 61, 45, 41, 32]
```

Let's divide these into bins of 18 to 25, 26 to 35, 36 to 60, and finally 61 and older. To do so, you have to use cut, a function in pandas:

```
In [76]: bins = [18, 25, 35, 60, 100]
```

```
In [77]: cats = pd.cut(ages, bins)
```

```
In [78]: cats
```

```
Out[78]:
```

```
[(18, 25], (18, 25], (18, 25], (25, 35], (18, 25], ..., (25, 35], (60, 100], (35, 60], (35, 60], (25, 35]]
```

```
Length: 12
```

```
Categories (4, interval[int64]): [(18, 25] < (25, 35] < (35, 60] < (60, 100]]
```

The object pandas returns is a special Categorical object. The output you see describes the bins computed by pandas.cut. You can treat it like an array of strings indicating the bin name; internally it contains a categories array specifying the distinct category names along with a labeling for the ages data in the codes attribute:

```
In [79]: cats.codes
```

```
Out[79]: array([0, 0, 0, 1, 0, 0, 2, 1, 3, 2, 2, 1], dtype=int8)
```

```
In [80]: cats.categories
```

```
Out[80]:
```

```
IntervalIndex([(18, 25], (25, 35], (35, 60], (60, 100]]
              closed='right',
              dtype='interval[int64]')
```

```
In [81]: pd.value_counts(cats)
```

```
Out[81]:
```

```
(18, 25]      5
(35, 60]      3
(25, 35]      3
(60, 100]     1
dtype: int64
```

Note that pd.value_counts(cats) are the bin counts for the result of pandas.cut.



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

Note that `pd.value_counts(cats)` are the bin counts for the result of `pandas.cut`.

Consistent with mathematical notation for intervals, a parenthesis means that the side is *open*, while the square bracket means it is *closed* (inclusive). You can change which side is closed by passing `right=False`:

```
In [82]: pd.cut(ages, [18, 26, 36, 61, 100], right=False)
Out[82]:
[[18, 26), [18, 26), [18, 26), [26, 36), [18, 26), ..., [26, 36), [61, 100), [36,
 61), [36, 61), [26, 36)]
Length: 12
Categories (4, interval[int64]): [[18, 26) < [26, 36) < [36, 61) < [61, 100)]
```

You can also pass your own bin names by passing a list or array to the `labels` option:

```
In [83]: group_names = ['Youth', 'YoungAdult', 'MiddleAged', 'Senior']

In [84]: pd.cut(ages, bins, labels=group_names)
Out[84]:
[Youth, Youth, Youth, YoungAdult, Youth, ..., YoungAdult, Senior, MiddleAged, Mid
dleAged, YoungAdult]
Length: 12
Categories (4, object): [Youth < YoungAdult < MiddleAged < Senior]
```

If you pass an integer number of bins to `cut` instead of explicit bin edges, it will compute equal-length bins based on the minimum and maximum values in the data. Consider the case of some uniformly distributed data chopped into fourths:

```
In [85]: data = np.random.rand(20)

In [86]: pd.cut(data, 4, precision=2)
Out[86]:
[(0.34, 0.55], (0.34, 0.55], (0.76, 0.97], (0.76, 0.97], (0.34, 0.55], ..., (0.34
, 0.55], (0.34, 0.55], (0.55, 0.76], (0.34, 0.55], (0.12, 0.34)]
Length: 20
Categories (4, interval[float64]): [(0.12, 0.34] < (0.34, 0.55] < (0.55, 0.76] <
(0.76, 0.97)]
```

The `precision=2` option limits the decimal precision to two digits.



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

A closely related function, `qcut`, bins the data based on sample quantiles. Depending on the distribution of the data, using `cut` will not usually result in each bin having the same number of data points. Since `qcut` uses sample quantiles instead, by definition you will obtain roughly equal-size bins:

```
In [87]: data = np.random.randn(1000) # Normally distributed

In [88]: cats = pd.qcut(data, 4) # Cut into quartiles

In [89]: cats
Out[89]:
[(-0.0265, 0.62], (0.62, 3.928], (-0.68, -0.0265], (0.62, 3.928], (-0.0265, 0.62]
, ..., (-0.68, -0.0265], (-0.68, -0.0265], (-2.95, -0.68], (0.62, 3.928], (-0.68,
-0.0265]]
Length: 1000
Categories (4, interval[float64]): [(-2.95, -0.68] < (-0.68, -0.0265] < (-0.0265,
0.62] <
(0.62, 3.928]]
```

```
In [90]: pd.value_counts(cats)
Out[90]:
(0.62, 3.928]      250
(-0.0265, 0.62]    250
(-0.68, -0.0265]   250
(-2.95, -0.68]     250
dtype: int64
```

Similar to `cut` you can pass your own quantiles (numbers between 0 and 1, inclusive):

```
In [91]: pd.qcut(data, [0, 0.1, 0.5, 0.9, 1.])
Out[91]:
[(-0.0265, 1.286], (-0.0265, 1.286], (-1.187, -0.0265], (-0.0265, 1.286], (-0.026
5, 1.286], ..., (-1.187, -0.0265], (-1.187, -0.0265], (-2.95, -1.187], (-0.0265,
1.286], (-1.187, -0.0265]]
Length: 1000
Categories (4, interval[float64]): [(-2.95, -1.187] < (-1.187, -0.0265] < (-0.026
5, 1.286] <
(1.286, 3.928]]
```

We'll return to `cut` and `qcut` later in the chapter during our discussion of aggregation and group operations, as these discretization functions are especially useful for quantile and group analysis.



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

Detecting and Filtering Outliers

Filtering or transforming outliers is largely a matter of applying array operations. Consider a DataFrame with some normally distributed data:

```
In [92]: data = pd.DataFrame(np.random.randn(1000, 4))

In [93]: data.describe()
Out[93]:
```

	0	1	2	3
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	0.049091	0.026112	-0.002544	-0.051827
std	0.996947	1.007458	0.995232	0.998311
min	-3.645860	-3.184377	-3.745356	-3.428254
25%	-0.599807	-0.612162	-0.687373	-0.747478
50%	0.047101	-0.013609	-0.022158	-0.088274
75%	0.756646	0.695298	0.699046	0.623331
max	2.653656	3.525865	2.735527	3.366626

Suppose you wanted to find values in one of the columns exceeding 3 in absolute value:

```
In [94]: col = data[2]

In [95]: col[np.abs(col) > 3]
Out[95]:
```

41	-3.399312
136	-3.745356

Name: 2, dtype: float64

To select all rows having a value exceeding 3 or -3, you can use the any method on a boolean DataFrame:

```
In [96]: data[(np.abs(data) > 3).any(1)]
Out[96]:
```

	0	1	2	3
41	0.457246	-0.025907	-3.399312	-0.974657
60	1.951312	3.260383	0.963301	1.201206
136	0.508391	-0.196713	-3.745356	-1.520113
235	-0.242459	-3.056990	1.918403	-0.578828
258	0.682841	0.326045	0.425384	-3.428254
322	1.179227	-3.184377	1.369891	-1.074833
544	-3.548824	1.553205	-2.186301	1.277104
635	-0.578093	0.193299	1.397822	3.366626
782	-0.207434	3.525865	0.283070	0.544635
803	-3.645860	0.255475	-0.549574	-1.907459



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

Values can be set based on these criteria. Here is code to cap values outside the interval -3 to 3:

```
In [97]: data[np.abs(data) > 3] = np.sign(data) * 3
```

```
In [98]: data.describe()
```

```
Out[98]:
```

	0	1	2	3
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	0.050286	0.025567	-0.001399	-0.051765
std	0.992920	1.004214	0.991414	0.995761
min	-3.000000	-3.000000	-3.000000	-3.000000
25%	-0.599807	-0.612162	-0.687373	-0.747478
50%	0.047101	-0.013609	-0.022158	-0.088274
75%	0.756646	0.695298	0.699046	0.623331
max	2.653656	3.000000	2.735527	3.000000

The statement `np.sign(data)` produces 1 and -1 values based on whether the values in data are positive or negative:

```
In [99]: np.sign(data).head()
```

```
Out[99]:
```

	0	1	2	3
0	-1.0	1.0	-1.0	1.0
1	1.0	-1.0	1.0	-1.0
2	1.0	1.0	1.0	-1.0
3	-1.0	-1.0	1.0	-1.0
4	-1.0	1.0	-1.0	-1.0



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

Permutation and Random Sampling

Permuting (randomly reordering) a Series or the rows in a DataFrame is easy to do using the `numpy.random.permutation` function. Calling permutation with the length of the axis you want to permute produces an array of integers indicating the new ordering:

```
In [100]: df = pd.DataFrame(np.arange(5 * 4).reshape((5, 4)))
```

```
In [101]: sampler = np.random.permutation(5)
```

```
In [102]: sampler
```

```
Out[102]: array([3, 1, 4, 2, 0])
```

That array can then be used in `iloc`-based indexing or the equivalent `take` function:

```
In [103]: df
```

```
Out[103]:
```

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15
4	16	17	18	19

```
In [104]: df.take(sampler)
```

```
Out[104]:
```

	0	1	2	3
3	12	13	14	15
1	4	5	6	7
4	16	17	18	19
2	8	9	10	11
0	0	1	2	3

To select a random subset without replacement, you can use the `sample` method on Series and DataFrame:

```
In [105]: df.sample(n=3)
```

```
Out[105]:
```

	0	1	2	3
3	12	13	14	15
4	16	17	18	19
2	8	9	10	11

To generate a sample *with* replacement (to allow repeat choices), pass `replace=True` to `sample`:

```
In [106]: choices = pd.Series([5, 7, -1, 6, 4])
```

```
In [107]: draws = choices.sample(n=10, replace=True)
```

```
In [108]: draws
```

```
Out[108]:
```

4	4
1	7
4	4
2	-1
0	5
3	6
4	4
0	5
4	4

```
dtype: int64
```




데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

Computing Indicator/Dummy Variables

Another type of transformation for statistical modeling or machine learning applications is converting a categorical variable into a “dummy” or “indicator” matrix. If a column in a DataFrame has k distinct values, you would derive a matrix or DataFrame with k columns containing all 1s and 0s. pandas has a `get_dummies` function for doing this, though devising one yourself is not difficult. Let's return to an earlier example DataFrame:

```
In [109]: df = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'b'],
.....:                      'data1': range(6)})
```

```
In [110]: pd.get_dummies(df['key'])
```

```
Out[110]:
```

	a	b	c
0	0	1	0
1	0	1	0
2	1	0	0
3	0	0	1
4	1	0	0
5	0	1	0

In some cases, you may want to add a prefix to the columns in the indicator DataFrame, which can then be merged with the other data. `get_dummies` has a `prefix` argument for doing this:

```
In [111]: dummies = pd.get_dummies(df['key'], prefix='key')
```

```
In [112]: df_with_dummy = df[['data1']].join(dummies)
```

```
In [113]: df_with_dummy
```

```
Out[113]:
```

	data1	key_a	key_b	key_c
0	0	0	1	0
1	1	0	1	0
2	2	1	0	0
3	3	0	0	1
4	4	1	0	0
5	5	0	1	0



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

If a row in a DataFrame belongs to multiple categories, things are a bit more complicated. Let's look at the MovieLens 1M dataset, which is investigated in more detail in

Chapter 14:

```
In [114]: mnames = ['movie_id', 'title', 'genres']

In [115]: movies = pd.read_table('datasets/movielens/movies.dat', sep='::',
.....:                          header=None, names=mnames)

In [116]: movies[:10]
Out[116]:
```

	movie_id	title	genres
0	1	Toy Story (1995)	Animation Children's Comedy
1	2	Jumanji (1995)	Adventure Children's Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama
4	5	Father of the Bride Part II (1995)	Comedy
5	6	Heat (1995)	Action Crime Thriller
6	7	Sabrina (1995)	Comedy Romance
7	8	Tom and Huck (1995)	Adventure Children's
8	9	Sudden Death (1995)	Action
9	10	GoldenEye (1995)	Action Adventure Thriller

Adding indicator variables for each genre requires a little bit of wrangling. First, we extract the list of unique genres in the dataset:

```
In [117]: all_genres = []

In [118]: for x in movies.genres:
.....:     all_genres.extend(x.split('|'))

In [119]: genres = pd.unique(all_genres)
```

Now we have:

```
In [120]: genres
Out[120]:
array(['Animation', 'Children's', 'Comedy', 'Adventure', 'Fantasy',
      'Romance', 'Drama', 'Action', 'Crime', 'Thriller', 'Horror',
      'Sci-Fi', 'Documentary', 'War', 'Musical', 'Mystery', 'Film-Noir',
      'Western'], dtype=object)
```

One way to construct the indicator DataFrame is to start with a DataFrame of all zeros:

```
In [121]: zero_matrix = np.zeros((len(movies), len(genres)))

In [122]: dummies = pd.DataFrame(zero_matrix, columns=genres)
```



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

Now, iterate through each movie and set entries in each row of dummies to 1. To do this, we use the `dummies.columns` to compute the column indices for each genre:

```
In [123]: gen = movies.genres[0]

In [124]: gen.split('|')
Out[124]: ['Animation', "Children's", 'Comedy']

In [125]: dummies.columns.get_indexer(gen.split('|'))
Out[125]: array([0, 1, 2])
```

Then, we can use `.iloc` to set values based on these indices:

```
In [126]: for i, gen in enumerate(movies.genres):
.....:     indices = dummies.columns.get_indexer(gen.split('|'))
.....:     dummies.iloc[i, indices] = 1
.....:
```

Then, as before, you can combine this with movies:

```
In [127]: movies_windic = movies.join(dummies.add_prefix('Genre_'))
```

```
In [128]: movies_windic.iloc[0]
Out[128]:
movie_id      1
title      Toy Story (1995)
genres      Animation|Children's|Comedy
Genre_Animation      1
Genre_Children's      1
Genre_Comedy      1
Genre_Adventure      0
Genre_Fantasy      0
Genre_Romance      0
Genre_Drama      0
...
Genre_Crime      0
Genre_Thriller      0
Genre_Horror      0
Genre_Sci-Fi      0
Genre_Documentary      0
Genre_War      0
Genre_Musical      0
Genre_Mystery      0
Genre_Film-Noir      0
Genre_Western      0
Name: 0, Length: 21, dtype: object
```



For much larger data, this method of constructing indicator variables with multiple membership is not especially speedy. It would be better to write a lower-level function that writes directly to a NumPy array, and then wrap the result in a DataFrame.



데이터 변형 (Filtering / Cleaning / Others)

Data Transformation

A useful recipe for statistical applications is to combine `get_dummies` with a discretization function like `cut`:

```
In [129]: np.random.seed(12345)

In [130]: values = np.random.rand(10)

In [131]: values
Out[131]:
array([ 0.9296,  0.3164,  0.1839,  0.2046,  0.5677,  0.5955,  0.9645,
        0.6532,  0.7489,  0.6536])

In [132]: bins = [0, 0.2, 0.4, 0.6, 0.8, 1]

In [133]: pd.get_dummies(pd.cut(values, bins))
Out[133]:
```

	(0.0, 0.2]	(0.2, 0.4]	(0.4, 0.6]	(0.6, 0.8]	(0.8, 1.0]
0	0	0	0	0	1
1	0	1	0	0	0
2	1	0	0	0	0
3	0	1	0	0	0
4	0	0	1	0	0
5	0	0	1	0	0
6	0	0	0	0	1
7	0	0	0	1	0
8	0	0	0	1	0
9	0	0	0	1	0

We set the random seed with `numpy.random.seed` to make the example deterministic.
We will look again at `pandas.get_dummies` later in the book.



문자열 다루기

String Manipulation

- Python, 부분적으로 문자열 및 텍스트 처리에 사용하기 쉽기 때문에 오랫동안 인기 있는 원천 데이터 조작 언어
- 대부분의 텍스트 작업은 string 객체의 내장 메서드를 사용하여 간단하게 해결!
- 보다 복잡한 패턴 일치 및 텍스트 조작의 경우 정규식이 필요
 - pandas는 문자열과 정규식을 전체 데이터 배열에 간결하게 적용할 수 있게 하여 추가로 누락된 데이터를 쉽게 처리!



String Object Methods

In many string munging and scripting applications, built-in string methods are sufficient. As an example, a comma-separated string can be broken into pieces with `split`:

```
In [134]: val = 'a,b, guido'

In [135]: val.split(',')
Out[135]: ['a', 'b', ' guido']
```

`split` is often combined with `strip` to trim whitespace (including line breaks):

```
In [136]: pieces = [x.strip() for x in val.split(',')]

In [137]: pieces
Out[137]: ['a', 'b', 'guido']
```

These substrings could be concatenated together with a two-colon delimiter using addition:

```
In [138]: first, second, third = pieces

In [139]: first + '::' + second + '::' + third
Out[139]: 'a::b::guido'
```

But this isn't a practical generic method. A faster and more Pythonic way is to pass a list or tuple to the `join` method on the string `::`:

```
In [140]: '::'.join(pieces)
Out[140]: 'a::b::guido'
```

Other methods are concerned with locating substrings. Using Python's `in` keyword is the best way to detect a substring, though `index` and `find` can also be used:

```
In [141]: 'guido' in val
Out[141]: True

In [142]: val.index(',')
Out[142]: 1

In [143]: val.find(',')
Out[143]: -1
```




문자열 다루기

String Manipulation

Note the difference between `find` and `index` is that `index` raises an exception if the string isn't found (versus returning `-1`):

```
In [144]: val.index(':')
-----
ValueError                                Traceback (most recent call last)
<ipython-input-144-280f8b2856ce> in <module>()
----> 1 val.index(':')
ValueError: substring not found
```

Relatedly, `count` returns the number of occurrences of a particular substring:

```
In [145]: val.count(',')
Out[145]: 2
```

`replace` will substitute occurrences of one pattern for another. It is commonly used to delete patterns, too, by passing an empty string:

```
In [146]: val.replace(',', ':')
Out[146]: 'a::b:: guido'

In [147]: val.replace(',', '')
Out[147]: 'ab guido'
```

See [Table 7-3](#) for a listing of some of Python's string methods.

Regular expressions can also be used with many of these operations, as you'll see.

Table 7-3. Python built-in string methods

Argument	Description
<code>count</code>	Return the number of non-overlapping occurrences of substring in the string.
<code>endswith</code>	Returns True if string ends with suffix.
<code>startswith</code>	Returns True if string starts with prefix.
<code>join</code>	Use string as delimiter for concatenating a sequence of other strings.
<code>index</code>	Return position of first character in substring if found in the string; raises <code>ValueError</code> if not found.
<code>find</code>	Return position of first character of first occurrence of substring in the string; like <code>index</code> , but returns <code>-1</code> if not found.
<code>rfind</code>	Return position of first character of last occurrence of substring in the string; returns <code>-1</code> if not found.
<code>replace</code>	Replace occurrences of string with another string.
<code>strip</code> , <code>rstrip</code> , <code>lstrip</code>	Trim whitespace, including newlines; equivalent to <code>x.strip()</code> (and <code>rstrip</code> , <code>lstrip</code> , respectively) for each element.
<code>split</code>	Break string into list of substrings using passed delimiter.
<code>lower</code>	Convert alphabet characters to lowercase.
<code>upper</code>	Convert alphabet characters to uppercase.
<code>casefold</code>	Convert characters to lowercase, and convert any region-specific variable character combinations to a common comparable form.
<code>ljust</code> , <code>rjust</code>	Left justify or right justify, respectively; pad opposite side of string with spaces (or some other fill character) to return a string with a minimum width.



Regular Expressions

Regular expressions provide a flexible way to search or match (often more complex) string patterns in text. A single expression, commonly called a *regex*, is a string formed according to the regular expression language. Python's built-in `re` module is responsible for applying regular expressions to strings; I'll give a number of examples of its use here.



The art of writing regular expressions could be a chapter of its own and thus is outside the book's scope. There are many excellent tutorials and references available on the internet and in other books.

The `re` module functions fall into three categories: pattern matching, substitution, and splitting. Naturally these are all related; a regex describes a pattern to locate in the text, which can then be used for many purposes. Let's look at a simple example: suppose we wanted to split a string with a variable number of whitespace characters (tabs, spaces, and newlines). The regex describing one or more whitespace characters is `\s+`:

```
In [148]: import re

In [149]: text = "foo    bar\t baz  \tqux"

In [150]: re.split('\s+', text)
Out[150]: ['foo', 'bar', 'baz', 'qux']
```

When you call `re.split('\s+', text)`, the regular expression is first *compiled*, and then its `split` method is called on the passed text. You can compile the regex yourself with `re.compile`, forming a reusable regex object:

```
In [151]: regex = re.compile('\s+')

In [152]: regex.split(text)
Out[152]: ['foo', 'bar', 'baz', 'qux']
```

If, instead, you wanted to get a list of all patterns matching the regex, you can use the `findall` method:

```
In [153]: regex.findall(text)
Out[153]: [' ', '\t ', ' ', '\t']
```



To avoid unwanted escaping with `\` in a regular expression, use *raw* string literals like `r'C:\x'` instead of the equivalent `'C:\\x'`.

Creating a regex object with `re.compile` is highly recommended if you intend to apply the same expression to many strings; doing so will save CPU cycles.



문자열 다루기

String Manipulation

`match` and `search` are closely related to `findall`. While `findall` returns all matches in a string, `search` returns only the first match. More rigidly, `match` *only* matches at the beginning of the string. As a less trivial example, let's consider a block of text and a regular expression capable of identifying most email addresses:

```
text = """Dave dave@google.com
Steve steve@gmail.com
Rob rob@gmail.com
Ryan ryan@yahoo.com
"""
pattern = r'[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,4}'

# re.IGNORECASE makes the regex case-insensitive
regex = re.compile(pattern, flags=re.IGNORECASE)
```

Using `findall` on the text produces a list of the email addresses:

```
In [155]: regex.findall(text)
Out[155]:
['dave@google.com',
'steve@gmail.com',
'rob@gmail.com',
'ryan@yahoo.com']
```

`search` returns a special match object for the first email address in the text. For the preceding regex, the match object can only tell us the start and end position of the pattern in the string:

```
In [156]: m = regex.search(text)

In [157]: m
Out[157]: <_sre.SRE_Match object; span=(5, 20), match='dave@google.com'>

In [158]: text[m.start():m.end()]
Out[158]: 'dave@google.com'
```

`regex.match` returns `None`, as it only will match if the pattern occurs at the start of the string:

```
In [159]: print(regex.match(text))
None
```

Relatedly, `sub` will return a new string with occurrences of the pattern replaced by the a new string:

```
In [160]: print(regex.sub('REDACTED', text))
Dave REDACTED
Steve REDACTED
Rob REDACTED
Ryan REDACTED
```



문자열 다루기

String Manipulation

Suppose you wanted to find email addresses and simultaneously segment each address into its three components: username, domain name, and domain suffix. To do this, put parentheses around the parts of the pattern to segment:

```
In [161]: pattern = r'([A-Z0-9._%+-]+)@([A-Z0-9.-]+\.[A-Z]{2,4})'
```

```
In [162]: regex = re.compile(pattern, flags=re.IGNORECASE)
```

A match object produced by this modified regex returns a tuple of the pattern components with its groups method:

```
In [163]: m = regex.match('wesm@bright.net')
```

```
In [164]: m.groups()
```

```
Out[164]: ('wesm', 'bright', 'net')
```

`findall` returns a list of tuples when the pattern has groups:

```
In [165]: regex.findall(text)
```

```
Out[165]:
```

```
[('dave', 'google', 'com'),  
 ('steve', 'gmail', 'com'),  
 ('rob', 'gmail', 'com'),  
 ('ryan', 'yahoo', 'com')]
```

`sub` also has access to groups in each match using special symbols like `\1` and `\2`. The symbol `\1` corresponds to the first matched group, `\2` corresponds to the second, and so forth:

```
In [166]: print(regex.sub(r'Username: \1, Domain: \2, Suffix: \3', text))
```

```
Dave Username: dave, Domain: google, Suffix: com
```

```
Steve Username: steve, Domain: gmail, Suffix: com
```

```
Rob Username: rob, Domain: gmail, Suffix: com
```

```
Ryan Username: ryan, Domain: yahoo, Suffix: com
```

There is much more to regular expressions in Python, most of which is outside the book's scope. [Table 7-4](#) provides a brief summary.

Table 7-4. Regular expression methods

Argument	Description
<code>findall</code>	Return all non-overlapping matching patterns in a string as a list
<code>finditer</code>	Like <code>findall</code> , but returns an iterator
<code>match</code>	Match pattern at start of string and optionally segment pattern components into groups; if the pattern matches, returns a match object, and otherwise <code>None</code>
<code>search</code>	Scan string for match to pattern; returning a match object if so; unlike <code>match</code> , the match can be anywhere in the string as opposed to only at the beginning
<code>split</code>	Break string into pieces at each occurrence of pattern
<code>sub</code> , <code>subn</code>	Replace all (<code>sub</code>) or first <code>n</code> occurrences (<code>subn</code>) of pattern in string with replacement expression; use symbols <code>\1</code> , <code>\2</code> , ... to refer to match group elements in the replacement string



Vectorized String Functions in pandas

Cleaning up a messy dataset for analysis often requires a lot of string munging and regularization. To complicate matters, a column containing strings will sometimes have missing data:

```
In [167]: data = {'Dave': 'dave@google.com', 'Steve': 'steve@gmail.com',  
.....:          'Rob': 'rob@gmail.com', 'Wes': np.nan}
```

```
In [168]: data = pd.Series(data)
```

```
In [169]: data
```

```
Out[169]:
```

```
Dave    dave@google.com  
Rob      rob@gmail.com  
Steve   steve@gmail.com  
Wes              NaN  
dtype: object
```

```
In [170]: data.isnull()
```

```
Out[170]:
```

```
Dave    False  
Rob      False  
Steve   False  
Wes      True  
dtype: bool
```

You can apply string and regular expression methods can be applied (passing a lambda or other function) to each value using `data.map`, but it will fail on the NA (null) values. To cope with this, Series has array-oriented methods for string operations that skip NA values. These are accessed through Series's `str` attribute; for example, we could check whether each email address has 'gmail' in it with `str.contains`:

```
In [171]: data.str.contains('gmail')
```

```
Out[171]:
```

```
Dave    False  
Rob      True  
Steve   True  
Wes      NaN  
dtype: object
```

Regular expressions can be used, too, along with any re options like `IGNORECASE`:

```
In [172]: pattern
```

```
Out[172]: '([A-Z0-9._%+-]+)@([A-Z0-9.-]+\.[A-Z]{2,4})'
```

```
In [173]: data.str.findall(pattern, flags=re.IGNORECASE)
```

```
Out[173]:
```

```
Dave    [(dave, google, com)]  
Rob      [(rob, gmail, com)]  
Steve   [(steve, gmail, com)]  
Wes              NaN  
dtype: object
```




문자열 다루기

String Manipulation

There are a couple of ways to do vectorized element retrieval. Either use `str.get` or index into the `str` attribute:

```
In [174]: matches = data.str.match(pattern, flags=re.IGNORECASE)
```

```
In [175]: matches
```

```
Out[175]:
```

```
Dave    True
Rob      True
Steve   True
Wes      NaN
dtype: object
```

To access elements in the embedded lists, we can pass an index to either of these functions:

```
In [176]: matches.str.get(1)
```

```
Out[176]:
```

```
Dave    NaN
Rob      NaN
Steve   NaN
Wes      NaN
dtype: float64
```

```
In [177]: matches.str[0]
```

```
Out[177]:
```

```
Dave    NaN
Rob      NaN
Steve   NaN
Wes      NaN
dtype: float64
```

You can similarly slice strings using this syntax:

```
In [178]: data.str[:5]
```

```
Out[178]:
```

```
Dave    dave@
Rob      rob@g
Steve    steve
Wes      NaN
dtype: object
```

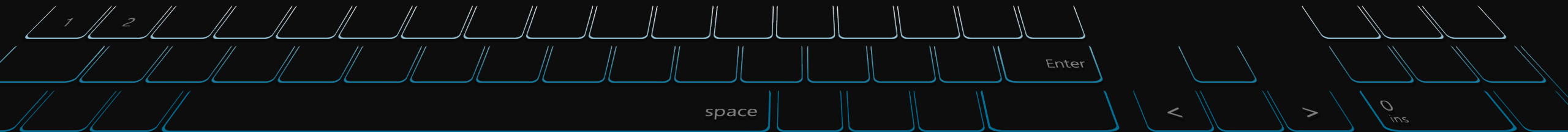



Table 7-5. Partial listing of vectorized string methods

Method	Description	Method	Description
cat	Concatenate strings element-wise with optional delimiter	match	Use <code>re.match</code> with the passed regular expression on each element, returning matched groups as list
contains	Return boolean array if each string contains pattern/regex	pad	Add whitespace to left, right, or both sides of strings
count	Count occurrences of pattern	center	Equivalent to <code>pad(side='both')</code>
extract	Use a regular expression with groups to extract one or more strings from a Series of strings; the result will be a DataFrame with one column per group	repeat	Duplicate values (e.g., <code>s.str.repeat(3)</code> is equivalent to <code>x * 3</code> for each string)
endswith	Equivalent to <code>x.endswith(pattern)</code> for each element	replace	Replace occurrences of pattern/regex with some other string
startswith	Equivalent to <code>x.startswith(pattern)</code> for each element	slice	Slice each string in the Series
findall	Compute list of all occurrences of pattern/regex for each string	split	Split strings on delimiter or regular expression
get	Index into each element (retrieve <i>i</i> -th element)	strip	Trim whitespace from both sides, including newlines
isalnum	Equivalent to built-in <code>str.alnum</code>	rstrip	Trim whitespace on right side
isalpha	Equivalent to built-in <code>str.isalpha</code>	lstrip	Trim whitespace on left side
isdecimal	Equivalent to built-in <code>str.isdecimal</code>		
isdigit	Equivalent to built-in <code>str.isdigit</code>		
islower	Equivalent to built-in <code>str.islower</code>		
isnumeric	Equivalent to built-in <code>str.isnumeric</code>		
isupper	Equivalent to built-in <code>str.isupper</code>		
join	Join strings in each element of the Series with passed separator		
len	Compute length of each string		
lower, upper	Convert cases; equivalent to <code>x.lower()</code> or <code>x.upper()</code> for each element		

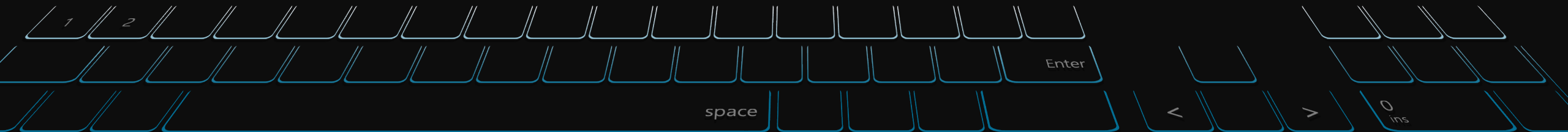
Q&A and Break Time

질의응답 및 휴식 시간 (5분)



2부: 데이터 정제 및 준비

- 1) 계층적 색인
- 2) 데이터 합치기
- 3) 재형성과 피벗

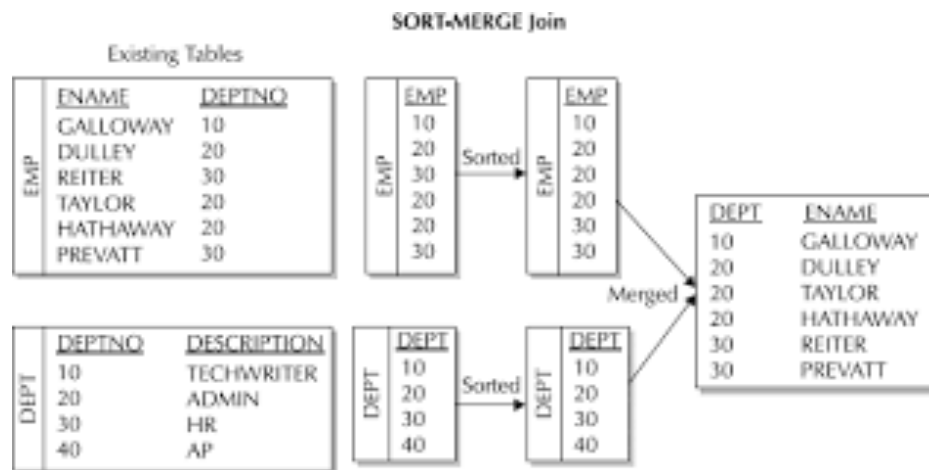
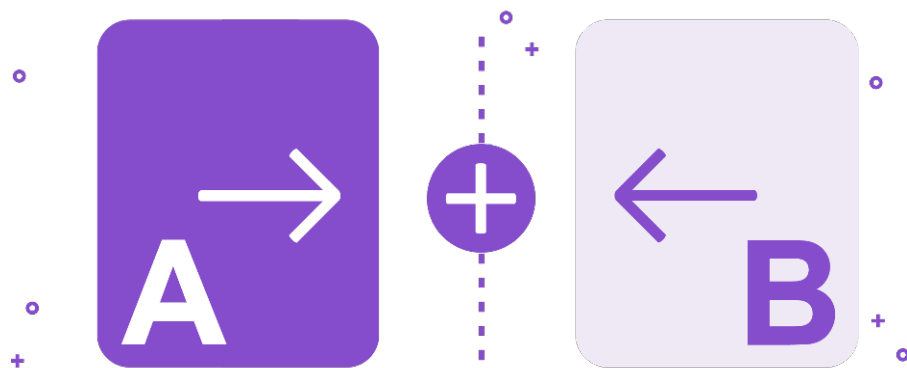




데이터 준비 (조인, 병합, 변형)

Data Wrangling: Join, Combine, and Reshape

- 데이터는,
많은 응용 프로그램에서 여러 파일 또는 데이터베이스에 분산되거나 분석하기 쉽지 않은 형식으로 배열
- 2부에서는 데이터를 조인, 결합 및 재정렬하는 데 도움이 되는 도구를 배움





8.1 Hierarchical Indexing

Hierarchical indexing is an important feature of pandas that enables you to have multiple (two or more) index levels on an axis. Somewhat abstractly, it provides a way for you to work with higher dimensional data in a lower dimensional form. Let's start with a simple example; create a Series with a list of lists (or arrays) as the index:

```
In [9]: data = pd.Series(np.random.randn(9),
...:                      index=[['a', 'a', 'a', 'b', 'b', 'c', 'c', 'd', 'd'],
...:                      [1, 2, 3, 1, 3, 1, 2, 2, 3]])
```

```
In [10]: data
Out[10]:
a 1 -0.204708
   2  0.478943
   3 -0.519439
b 1 -0.555730
   3  1.965781
c 1  1.393406
   2  0.092908
   3  0.769023
dtype: float64
```

What you're seeing is a prettified view of a Series with a `MultiIndex` as its index. The "gaps" in the index display mean "use the label directly above":

```
In [11]: data.index
Out[11]:
MultiIndex(levels=[['a', 'b', 'c', 'd'], [1, 2, 3]],
            labels=[[0, 0, 0, 1, 1, 2, 2, 3, 3], [0, 1, 2, 0, 2, 0, 1, 1, 2]])
```

With a hierarchically indexed object, so-called *partial* indexing is possible, enabling you to concisely select subsets of the data:

```
In [12]: data['b']
Out[12]:
1 -0.555730
3  1.965781
dtype: float64
```

```
In [13]: data['b':'c']
Out[13]:
b 1 -0.555730
   3  1.965781
c 1  1.393406
   2  0.092908
dtype: float64
```

```
In [14]: data.loc[['b', 'd']]
Out[14]:
b 1 -0.555730
   3  1.965781
d 2  0.281746
   3  0.769023
dtype: float64
```



Selection is even possible from an “inner” level:

```
In [15]: data.loc[:, 2]
Out[15]:
a    0.478943
c    0.092908
d    0.281746
dtype: float64
```

Hierarchical indexing plays an important role in reshaping data and group-based operations like forming a pivot table. For example, you could rearrange the data into a DataFrame using its unstack method:

```
In [16]: data.unstack()
Out[16]:
```

	1	2	3
a	-0.204708	0.478943	-0.519439
b	-0.555730	NaN	1.965781
c	1.393406	0.092908	NaN
d	NaN	0.281746	0.769023

The inverse operation of unstack is stack:

```
In [17]: data.unstack().stack()
Out[17]:
a 1 -0.204708
  2  0.478943
  3 -0.519439
b 1 -0.555730
  3  1.965781
c 1  1.393406
  2  0.092908
d 2  0.281746
  3  0.769023
dtype: float64
```

stack and unstack will be explored in more detail later in this chapter.



계층적 색인

Hierarchical Indexing

With a DataFrame, either axis can have a hierarchical index:

```
In [18]: frame = pd.DataFrame(np.arange(12).reshape((4, 3)),
.....:                        index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]],
.....:                        columns=[['Ohio', 'Ohio', 'Colorado'],
.....:                                ['Green', 'Red', 'Green']])
```

In [19]: frame

```
Out[19]:
```

		Ohio		Colorado
		Green	Red	Green
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

The hierarchical levels can have names (as strings or any Python objects). If so, these will show up in the console output:

```
In [20]: frame.index.names = ['key1', 'key2']
```

```
In [21]: frame.columns.names = ['state', 'color']
```

In [22]: frame

```
Out[22]:
```

		state		Ohio		Colorado
			color	Green	Red	Green
		key1	key2			
a	1			0	1	2
	2			3	4	5
b	1			6	7	8
	2			9	10	11



Be careful to distinguish the index names 'state' and 'color' from the row labels.

With partial column indexing you can similarly select groups of columns:

```
In [23]: frame['Ohio']
```

```
Out[23]:
```

		color		Green	Red
		key1	key2		
a	1			0	1
	2			3	4
b	1			6	7
	2			9	10

A MultiIndex can be created by itself and then reused; the columns in the preceding DataFrame with level names could be created like this:

```
MultiIndex.from_arrays([['Ohio', 'Ohio', 'Colorado'], ['Green', 'Red', 'Green']],
                      names=['state', 'color'])
```



Reordering and Sorting Levels

At times you will need to rearrange the order of the levels on an axis or sort the data by the values in one specific level. The `swaplevel` takes two level numbers or names and returns a new object with the levels interchanged (but the data is otherwise unaltered):

```
In [24]: frame.swaplevel('key1', 'key2')
```

```
Out[24]:
```

state		Ohio		Colorado
color		Green	Red	Green
key2	key1			
1	a	0	1	2
2	a	3	4	5
1	b	6	7	8
2	b	9	10	11

`sort_index`, on the other hand, sorts the data using only the values in a single level. When swapping levels, it's not uncommon to also use `sort_index` so that the result is lexicographically sorted by the indicated level:

```
In [25]: frame.sort_index(level=1)
```

```
Out[25]:
```

state		Ohio		Colorado
color		Green	Red	Green
key1	key2			
a	1	0	1	2
b	1	6	7	8
a	2	3	4	5
b	2	9	10	11

```
In [26]: frame.swaplevel(0, 1).sort_index(level=0)
```

```
Out[26]:
```

state		Ohio		Colorado
color		Green	Red	Green
key2	key1			
1	a	0	1	2
	b	6	7	8
2	a	3	4	5
	b	9	10	11



Data selection performance is much better on hierarchically indexed objects if the index is lexicographically sorted starting with the outermost level—that is, the result of calling `sort_index(level=0)` or `sort_index()`.



Summary Statistics by Level

Many descriptive and summary statistics on DataFrame and Series have a `level` option in which you can specify the level you want to aggregate by on a particular axis. Consider the above DataFrame; we can aggregate by level on either the rows or columns like so:

```
In [27]: frame.sum(level='key2')
```

```
Out[27]:
```

```
state  Ohio      Colorado  
color  Green Red      Green  
key2
```

```
1         6   8         10  
2        12  14         16
```

```
In [28]: frame.sum(level='color', axis=1)
```

```
Out[28]:
```

```
color      Green  Red  
key1 key2
```

```
a     1         2     1  
      2         8     4  
b     1        14     7  
      2        20    10
```

Under the hood, this utilizes pandas's groupby machinery, which will be discussed in more detail later in the book.



Indexing with a DataFrame's columns

It's not unusual to want to use one or more columns from a DataFrame as the row index; alternatively, you may wish to move the row index into the DataFrame's columns. Here's an example DataFrame:

```
In [29]: frame = pd.DataFrame({'a': range(7), 'b': range(7, 0, -1),
.....:                        'c': ['one', 'one', 'one', 'two', 'two',
.....:                              'two', 'two'],
.....:                        'd': [0, 1, 2, 0, 1, 2, 3]})
```

```
In [30]: frame
```

```
Out[30]:
```

	a	b	c	d
0	0	7	one	0
1	1	6	one	1
2	2	5	one	2
3	3	4	two	0
4	4	3	two	1
5	5	2	two	2
6	6	1	two	3

DataFrame's `set_index` function will create a new DataFrame using one or more of its columns as the index:

```
In [31]: frame2 = frame.set_index(['c', 'd'])
```

```
In [32]: frame2
```

```
Out[32]:
```

	a	b
one	0	7
	1	6
	2	5
two	0	4
	1	3
	2	2
	3	1

By default the columns are removed from the DataFrame, though you can leave them in:

```
In [33]: frame.set_index(['c', 'd'], drop=False)
```

```
Out[33]:
```

	a	b	c	d
one	0	7	one	0
	1	6	one	1
	2	5	one	2
two	0	4	two	0
	1	3	two	1
	2	2	two	2
	3	1	two	3



계층적 색인

Hierarchical Indexing

`reset_index`, on the other hand, does the opposite of `set_index`; the hierarchical index levels are moved into the columns:

```
In [34]: frame2.reset_index()
```

```
Out[34]:
```

	c	d	a	b
0	one	0	0	7
1	one	1	1	6
2	one	2	2	5
3	two	0	3	4
4	two	1	4	3
5	two	2	5	2
6	two	3	6	1



8.2 Combining and Merging Datasets

Data contained in pandas objects can be combined together in a number of ways:

- `pandas.merge` connects rows in DataFrames based on one or more keys. This will be familiar to users of SQL or other relational databases, as it implements database *join* operations.
- `pandas.concat` concatenates or “stacks” together objects along an axis.
- The `combine_first` instance method enables splicing together overlapping data to fill in missing values in one object with values from another.

I will address each of these and give a number of examples. They’ll be utilized in examples throughout the rest of the book.



Database-Style DataFrame Joins

Merge or *join* operations combine datasets by linking rows using one or more *keys*. These operations are central to relational databases (e.g., SQL-based). The `merge` function in pandas is the main entry point for using these algorithms on your data.

Let's start with a simple example:

```
In [35]: df1 = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'a', 'b'],
.....:                      'data1': range(7)})
```

```
In [36]: df2 = pd.DataFrame({'key': ['a', 'b', 'd'],
.....:                      'data2': range(3)})
```

```
In [37]: df1
```

```
Out[37]:
```

	data1	key
0	0	b
1	1	b
2	2	a
3	3	c
4	4	a
5	5	a
6	6	b

```
In [38]: df2
```

```
Out[38]:
```

	data2	key
0	0	a
1	1	b
2	2	d

This is an example of a *many-to-one* join; the data in `df1` has multiple rows labeled a and b, whereas `df2` has only one row for each value in the key column. Calling `merge` with these objects we obtain:

```
In [39]: pd.merge(df1, df2)
```

```
Out[39]:
```

	data1	key	data2
0	0	b	1
1	1	b	1
2	6	b	1
3	2	a	0
4	4	a	0
5	5	a	0

Note that I didn't specify which column to join on. If that information is not specified, `merge` uses the overlapping column names as the keys. It's a good practice to specify explicitly, though:

```
In [40]: pd.merge(df1, df2, on='key')
```

```
Out[40]:
```

	data1	key	data2
0	0	b	1
1	1	b	1
2	6	b	1
3	2	a	0
4	4	a	0
5	5	a	0



데이터 합치기

Combining and Merging Datasets

If the column names are different in each object, you can specify them separately:

```
In [41]: df3 = pd.DataFrame({'lkey': ['b', 'b', 'a', 'c', 'a', 'a', 'b'],
.....:                      'data1': range(7)})
```

```
In [42]: df4 = pd.DataFrame({'rkey': ['a', 'b', 'd'],
.....:                      'data2': range(3)})
```

```
In [43]: pd.merge(df3, df4, left_on='lkey', right_on='rkey')
```

```
Out[43]:
  data1 lkey data2 rkey
0      0    b      1    b
1      1    b      1    b
2      6    b      1    b
3      2    a      0    a
4      4    a      0    a
5      5    a      0    a
```

You may notice that the 'c' and 'd' values and associated data are missing from the result. By default merge does an 'inner' join; the keys in the result are the intersection, or the common set found in both tables. Other possible options are 'left', 'right', and 'outer'. The outer join takes the union of the keys, combining the effect of applying both left and right joins:

```
In [44]: pd.merge(df1, df2, how='outer')
```

```
Out[44]:
  data1 key data2
0      0.0  b   1.0
1      1.0  b   1.0
2      6.0  b   1.0
3      2.0  a   0.0
4      4.0  a   0.0
5      5.0  a   0.0
6      3.0  c   NaN
7      NaN  d   2.0
```

See [Table 8-1](#) for a summary of the options for how.

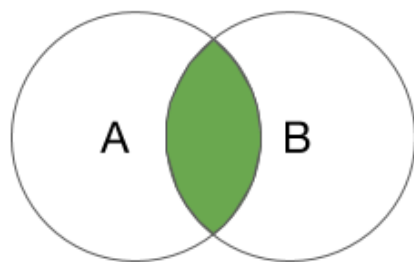
Table 8-1. Different join types with how argument

Option	Behavior
'inner'	Use only the key combinations observed in both tables
'left'	Use all key combinations found in the left table
'right'	Use all key combinations found in the right table
'outer'	Use all key combinations observed in both tables together

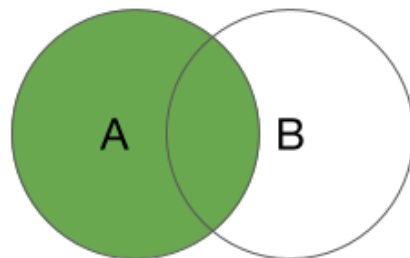


데이터 합치기

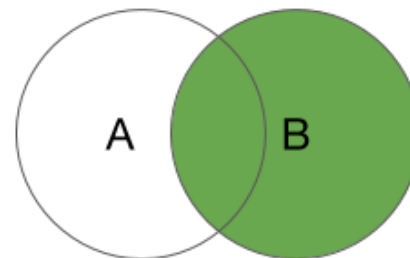
Combining and Merging Datasets



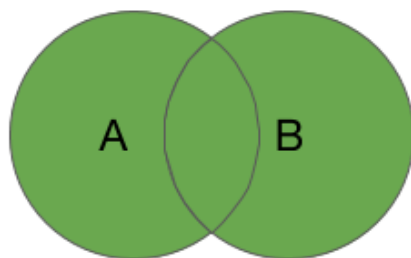
INNER JOIN



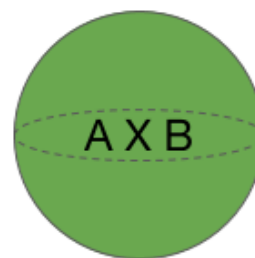
LEFT OUTER JOIN



RIGHT OUTER
JOIN



FULL OUTER
JOIN



CARTESIAN
(CROSS) JOIN



데이터 합치기

Combining and Merging Datasets

Many-to-many merges have well-defined, though not necessarily intuitive, behavior.

Here's an example:

```
In [45]: df1 = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'b'],
.....:                      'data1': range(6)})
```

```
In [46]: df2 = pd.DataFrame({'key': ['a', 'b', 'a', 'b', 'd'],
.....:                      'data2': range(5)})
```

```
In [47]: df1
```

```
Out[47]:
```

	data1	key
0	0	b
1	1	b
2	2	a
3	3	c
4	4	a
5	5	b

```
In [48]: df2
```

```
Out[48]:
```

	data2	key
0	0	a
1	1	b
2	2	a
3	3	b
4	4	d

```
In [49]: pd.merge(df1, df2, on='key', how='left')
```

```
Out[49]:
```

	data1	key	data2
0	0	b	1.0
1	0	b	3.0
2	1	b	1.0
3	1	b	3.0
4	2	a	0.0
5	2	a	2.0
6	3	c	NaN
7	4	a	0.0
8	4	a	2.0
9	5	b	1.0
10	5	b	3.0

Many-to-many joins form the Cartesian product of the rows. Since there were three 'b' rows in the left DataFrame and two in the right one, there are six 'b' rows in the result. The join method only affects the distinct key values appearing in the result:

```
In [50]: pd.merge(df1, df2, how='inner')
```

```
Out[50]:
```

	data1	key	data2
0	0	b	1
1	0	b	3
2	1	b	1
3	1	b	3
4	5	b	1
5	5	b	3
6	2	a	0
7	2	a	2
8	4	a	0
9	4	a	2



데이터 합치기

Combining and Merging Datasets

To merge with multiple keys, pass a list of column names:

```
In [51]: left = pd.DataFrame({'key1': ['foo', 'foo', 'bar'],
....:                        'key2': ['one', 'two', 'one'],
....:                        'lval': [1, 2, 3]})

In [52]: right = pd.DataFrame({'key1': ['foo', 'foo', 'bar', 'bar'],
....:                          'key2': ['one', 'one', 'one', 'two'],
....:                          'rval': [4, 5, 6, 7]})

In [53]: pd.merge(left, right, on=['key1', 'key2'], how='outer')

Out[53]:
```

	key1	key2	lval	rval
0	foo	one	1.0	4.0
1	foo	one	1.0	5.0
2	foo	two	2.0	NaN
3	bar	one	3.0	6.0
4	bar	two	NaN	7.0

To determine which key combinations will appear in the result depending on the choice of merge method, think of the multiple keys as forming an array of tuples to be used as a single join key (even though it's not actually implemented that way).



When you're joining columns-on-columns, the indexes on the passed DataFrame objects are discarded.

A last issue to consider in merge operations is the treatment of overlapping column names. While you can address the overlap manually (see the earlier section on renaming axis labels), merge has a `suffixes` option for specifying strings to append to overlapping names in the left and right DataFrame objects:

```
In [54]: pd.merge(left, right, on='key1')
Out[54]:
```

	key1	key2_x	lval	key2_y	rval
0	foo	one	1	one	4
1	foo	one	1	one	5
2	foo	two	2	one	4
3	foo	two	2	one	5
4	bar	one	3	one	6
5	bar	one	3	two	7

```
In [55]: pd.merge(left, right, on='key1', suffixes=('_left', '_right'))
Out[55]:
```

	key1	key2_left	lval	key2_right	rval
0	foo	one	1	one	4
1	foo	one	1	one	5
2	foo	two	2	one	4
3	foo	two	2	one	5
4	bar	one	3	one	6
5	bar	one	3	two	7



데이터 합치기

Combining and Merging Datasets

Table 8-2. merge function arguments

Argument	Description
left	DataFrame to be merged on the left side.
right	DataFrame to be merged on the right side.
how	One of 'inner', 'outer', 'left', or 'right'; defaults to 'inner'.
on	Column names to join on. Must be found in both DataFrame objects. If not specified and no other join keys given, will use the intersection of the column names in left and right as the join keys.
left_on	Columns in left DataFrame to use as join keys.
right_on	Analogous to left_on for right DataFrame.
left_index	Use row index in left as its join key (or keys, if a MultiIndex).
right_index	Analogous to left_index.
sort	Sort merged data lexicographically by join keys; True by default (disable to get better performance in some cases on large datasets).
suffixes	Tuple of string values to append to column names in case of overlap; defaults to ('_x', '_y') (e.g., if 'data' in both DataFrame objects, would appear as 'data_x' and 'data_y' in result).
copy	If False, avoid copying data into resulting data structure in some exceptional cases; by default always copies.
indicator	Adds a special column _merge that indicates the source of each row; values will be 'left_only', 'right_only', or 'both' based on the origin of the joined data in each row.



Merging on Index

In some cases, the merge key(s) in a DataFrame will be found in its index. In this case, you can pass `left_index=True` or `right_index=True` (or both) to indicate that the index should be used as the merge key:

```
In [56]: left1 = pd.DataFrame({'key': ['a', 'b', 'a', 'a', 'b', 'c'],  
.....:                      'value': range(6)})  
  
In [57]: right1 = pd.DataFrame({'group_val': [3.5, 7]}, index=['a', 'b'])
```

```
In [58]: left1
```

```
Out[58]:
```

	key	value
0	a	0
1	b	1
2	a	2
3	a	3
4	b	4
5	c	5

```
In [59]: right1
```

```
Out[59]:
```

	group_val
a	3.5
b	7.0

```
In [60]: pd.merge(left1, right1, left_on='key', right_index=True)
```

```
Out[60]:
```

	key	value	group_val
0	a	0	3.5
2	a	2	3.5
3	a	3	3.5
1	b	1	7.0
4	b	4	7.0

Since the default merge method is to intersect the join keys, you can instead form the union of them with an outer join:

```
In [61]: pd.merge(left1, right1, left_on='key', right_index=True, how='outer')
```

```
Out[61]:
```

	key	value	group_val
0	a	0	3.5
2	a	2	3.5
3	a	3	3.5
1	b	1	7.0
4	b	4	7.0
5	c	5	NaN



데이터 합치기

Combining and Merging Datasets

With hierarchically indexed data, things are more complicated, as joining on index is implicitly a multiple-key merge:

```
In [62]: lefth = pd.DataFrame({'key1': ['Ohio', 'Ohio', 'Ohio',
.....:                               'Nevada', 'Nevada'],
.....:                       'key2': [2000, 2001, 2002, 2001, 2002],
.....:                       'data': np.arange(5.)})

In [63]: righth = pd.DataFrame(np.arange(12).reshape((6, 2)),
.....:                          index=[['Nevada', 'Nevada', 'Ohio', 'Ohio',
.....:                               'Ohio', 'Ohio'],
.....:                          [2001, 2000, 2000, 2000, 2001, 2002]],
.....:                          columns=['event1', 'event2'])
```

```
In [64]: lefth
Out[64]:
```

	data	key1	key2
0	0.0	Ohio	2000
1	1.0	Ohio	2001
2	2.0	Ohio	2002
3	3.0	Nevada	2001
4	4.0	Nevada	2002

```
In [65]: righth
Out[65]:
```

		event1	event2
Nevada	2001	0	1
	2000	2	3
Ohio	2000	4	5
	2000	6	7
	2001	8	9
	2002	10	11

In this case, you have to indicate multiple columns to merge on as a list (note the handling of duplicate index values with how='outer'):

```
In [66]: pd.merge(lefth, righth, left_on=['key1', 'key2'], right_index=True)
Out[66]:
```

	data	key1	key2	event1	event2
0	0.0	Ohio	2000	4	5
0	0.0	Ohio	2000	6	7
1	1.0	Ohio	2001	8	9
2	2.0	Ohio	2002	10	11
3	3.0	Nevada	2001	0	1

```
In [67]: pd.merge(lefth, righth, left_on=['key1', 'key2'],
.....:             right_index=True, how='outer')
Out[67]:
```

	data	key1	key2	event1	event2
0	0.0	Ohio	2000	4.0	5.0
0	0.0	Ohio	2000	6.0	7.0
1	1.0	Ohio	2001	8.0	9.0
2	2.0	Ohio	2002	10.0	11.0
3	3.0	Nevada	2001	0.0	1.0
4	4.0	Nevada	2002	NaN	NaN
4	NaN	Nevada	2000	2.0	3.0



데이터 합치기

Combining and Merging Datasets

Using the indexes of both sides of the merge is also possible:

```
In [68]: left2 = pd.DataFrame([[1., 2.], [3., 4.], [5., 6.]],
.....:                        index=['a', 'c', 'e'],
.....:                        columns=['Ohio', 'Nevada'])

In [69]: right2 = pd.DataFrame([[7., 8.], [9., 10.], [11., 12.], [13., 14.]],
.....:                         index=['b', 'c', 'd', 'e'],
.....:                         columns=['Missouri', 'Alabama'])
```

```
In [70]: left2
Out[70]:
```

	Ohio	Nevada
a	1.0	2.0
c	3.0	4.0
e	5.0	6.0

```
In [71]: right2
Out[71]:
```

	Missouri	Alabama
b	7.0	8.0
c	9.0	10.0
d	11.0	12.0
e	13.0	14.0

```
In [72]: pd.merge(left2, right2, how='outer', left_index=True, right_index=True)
```

```
Out[72]:
```

	Ohio	Nevada	Missouri	Alabama
a	1.0	2.0	NaN	NaN
b	NaN	NaN	7.0	8.0
c	3.0	4.0	9.0	10.0
d	NaN	NaN	11.0	12.0
e	5.0	6.0	13.0	14.0

DataFrame has a convenient join instance for merging by index. It can also be used to combine together many DataFrame objects having the same or similar indexes but non-overlapping columns. In the prior example, we could have written:

```
In [73]: left2.join(right2, how='outer')
```

```
Out[73]:
```

	Ohio	Nevada	Missouri	Alabama
a	1.0	2.0	NaN	NaN
b	NaN	NaN	7.0	8.0
c	3.0	4.0	9.0	10.0
d	NaN	NaN	11.0	12.0
e	5.0	6.0	13.0	14.0



데이터 합치기

Combining and Merging Datasets

In part for legacy reasons (i.e., much earlier versions of pandas), DataFrame's `join` method performs a left join on the join keys, exactly preserving the left frame's row index. It also supports joining the index of the passed DataFrame on one of the columns of the calling DataFrame:

```
In [74]: left1.join(right1, on='key')
Out[74]:
```

	key	value	group_val
0	a	0	3.5
1	b	1	7.0
2	a	2	3.5
3	a	3	3.5
4	b	4	7.0
5	c	5	NaN

Lastly, for simple index-on-index merges, you can pass a list of DataFrames to join as an alternative to using the more general `concat` function described in the next section:

```
In [75]: another = pd.DataFrame([[7., 8.], [9., 10.], [11., 12.], [16., 17.]],
.....:                          index=['a', 'c', 'e', 'f'],
.....:                          columns=['New York', 'Oregon'])
```

```
In [76]: another
Out[76]:
```

	New York	Oregon
a	7.0	8.0
c	9.0	10.0
e	11.0	12.0
f	16.0	17.0

```
In [77]: left2.join([right2, another])
Out[77]:
```

	Ohio	Nevada	Missouri	Alabama	New York	Oregon
a	1.0	2.0	NaN	NaN	7.0	8.0
c	3.0	4.0	9.0	10.0	9.0	10.0
e	5.0	6.0	13.0	14.0	11.0	12.0

```
In [78]: left2.join([right2, another], how='outer')
Out[78]:
```

	Ohio	Nevada	Missouri	Alabama	New York	Oregon
a	1.0	2.0	NaN	NaN	7.0	8.0
b	NaN	NaN	7.0	8.0	NaN	NaN
c	3.0	4.0	9.0	10.0	9.0	10.0
d	NaN	NaN	11.0	12.0	NaN	NaN
e	5.0	6.0	13.0	14.0	11.0	12.0
f	NaN	NaN	NaN	NaN	16.0	17.0



Concatenating Along an Axis

Another kind of data combination operation is referred to interchangeably as concatenation, binding, or stacking. NumPy's `concatenate` function can do this with NumPy arrays:

```
In [79]: arr = np.arange(12).reshape((3, 4))
```

```
In [80]: arr
```

```
Out[80]:  
array([[ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11]])
```

```
In [81]: np.concatenate([arr, arr], axis=1)
```

```
Out[81]:  
array([[ 0,  1,  2,  3,  0,  1,  2,  3],  
       [ 4,  5,  6,  7,  4,  5,  6,  7],  
       [ 8,  9, 10, 11,  8,  9, 10, 11]])
```

In the context of pandas objects such as `Series` and `DataFrame`, having labeled axes enable you to further generalize array concatenation. In particular, you have a number of additional things to think about:

- If the objects are indexed differently on the other axes, should we combine the distinct elements in these axes or use only the shared values (the intersection)?
- Do the concatenated chunks of data need to be identifiable in the resulting object?
- Does the “concatenation axis” contain data that needs to be preserved? In many cases, the default integer labels in a `DataFrame` are best discarded during concatenation.



데이터 합치기

Combining and Merging Datasets

The `concat` function in pandas provides a consistent way to address each of these concerns. I'll give a number of examples to illustrate how it works. Suppose we have three Series with no index overlap:

```
In [82]: s1 = pd.Series([0, 1], index=['a', 'b'])
```

```
In [83]: s2 = pd.Series([2, 3, 4], index=['c', 'd', 'e'])
```

```
In [84]: s3 = pd.Series([5, 6], index=['f', 'g'])
```

Calling `concat` with these objects in a list glues together the values and indexes:

```
In [85]: pd.concat([s1, s2, s3])
```

```
Out[85]:  
a    0  
b    1  
c    2  
d    3  
e    4  
f    5  
g    6  
dtype: int64
```

By default `concat` works along `axis=0`, producing another Series. If you pass `axis=1`, the result will instead be a DataFrame (`axis=1` is the columns):

```
In [86]: pd.concat([s1, s2, s3], axis=1)
```

```
Out[86]:  
      0    1    2  
a  0.0  NaN  NaN  
b  1.0  NaN  NaN  
c  NaN  2.0  NaN  
d  NaN  3.0  NaN  
e  NaN  4.0  NaN  
f  NaN  NaN  5.0  
g  NaN  NaN  6.0
```




데이터 합치기

Combining and Merging Datasets

In this case there is no overlap on the other axis, which as you can see is the sorted union (the 'outer' join) of the indexes. You can instead intersect them by passing `join='inner'`:

```
In [87]: s4 = pd.concat([s1, s3])
```

```
In [88]: s4
```

```
Out[88]:
```

```
a    0
b    1
f    5
g    6
dtype: int64
```

```
In [89]: pd.concat([s1, s4], axis=1)
```

```
   0  1
a  0.0  0
b  1.0  1
f  NaN  5
g  NaN  6
```

```
In [90]: pd.concat([s1, s4], axis=1, join='inner')
```

```
Out[90]:
   0  1
a  0  0
b  1  1
```

In this last example, the 'f' and 'g' labels disappeared because of the `join='inner'` option.

You can even specify the axes to be used on the other axes with `join_axes`:

```
In [91]: pd.concat([s1, s4], axis=1, join_axes=[['a', 'c', 'b', 'e']])
```

```
Out[91]:
```

```
   0  1
a  0.0  0.0
c  NaN  NaN
b  1.0  1.0
e  NaN  NaN
```

A potential issue is that the concatenated pieces are not identifiable in the result. Suppose instead you wanted to create a hierarchical index on the concatenation axis. To do this, use the `keys` argument:

```
In [92]: result = pd.concat([s1, s1, s3], keys=['one', 'two', 'three'])
```

```
In [93]: result
```

```
Out[93]:
```

```
one   a    0
      b    1
two   a    0
      b    1
three f    5
      g    6
```

```
dtype: int64
```

```
In [94]: result.unstack()
```

```
Out[94]:
```

```
      a    b    f    g
one   0.0  1.0  NaN  NaN
two   0.0  1.0  NaN  NaN
three NaN  NaN  5.0  6.0
```



데이터 합치기

Combining and Merging Datasets

In the case of combining Series along `axis=1`, the keys become the DataFrame column headers:

```
In [95]: pd.concat([s1, s2, s3], axis=1, keys=['one', 'two', 'three'])
```

```
Out[95]:
```

	one	two	three
a	0.0	NaN	NaN
b	1.0	NaN	NaN
c	NaN	2.0	NaN
d	NaN	3.0	NaN
e	NaN	4.0	NaN
f	NaN	NaN	5.0
g	NaN	NaN	6.0

The same logic extends to DataFrame objects:

```
In [96]: df1 = pd.DataFrame(np.arange(6).reshape(3, 2), index=['a', 'b', 'c'],  
.....:                      columns=['one', 'two'])
```

```
In [97]: df2 = pd.DataFrame(5 + np.arange(4).reshape(2, 2), index=['a', 'c'],  
.....:                      columns=['three', 'four'])
```

```
In [98]: df1
```

```
Out[98]:
```

	one	two
a	0	1
b	2	3
c	4	5

```
In [99]: df2
```

```
Out[99]:
```

	three	four
a	5	6
c	7	8

```
In [100]: pd.concat([df1, df2], axis=1, keys=['level1', 'level2'])
```

```
Out[100]:
```

	level1		level2	
	one	two	three	four
a	0	1	5.0	6.0
b	2	3	NaN	NaN
c	4	5	7.0	8.0



데이터 합치기

Combining and Merging Datasets

If you pass a dict of objects instead of a list, the dict's keys will be used for the keys option:

```
In [101]: pd.concat({'level1': df1, 'level2': df2}, axis=1)
Out[101]:
```

	level1		level2	
	one	two	three	four
a	0	1	5.0	6.0
b	2	3	NaN	NaN
c	4	5	7.0	8.0

There are additional arguments governing how the hierarchical index is created (see Table 8-3). For example, we can name the created axis levels with the names argument:

```
In [102]: pd.concat([df1, df2], axis=1, keys=['level1', 'level2'],
.....:               names=['upper', 'lower'])
Out[102]:
```

	upper level1		level2		
	lower	one	two	three	four
a		0	1	5.0	6.0
b		2	3	NaN	NaN
c		4	5	7.0	8.0

A last consideration concerns DataFrames in which the row index does not contain any relevant data:

```
In [103]: df1 = pd.DataFrame(np.random.randn(3, 4), columns=['a', 'b', 'c', 'd'])
```

```
In [104]: df2 = pd.DataFrame(np.random.randn(2, 3), columns=['b', 'd', 'a'])
```

```
In [105]: df1
Out[105]:
```

	a	b	c	d
0	1.246435	1.007189	-1.296221	0.274992
1	0.228913	1.352917	0.886429	-2.001637
2	-0.371843	1.669025	-0.438570	-0.539741

```
In [106]: df2
Out[106]:
```

	b	d	a
0	0.476985	3.248944	-1.021228
1	-0.577087	0.124121	0.302614

In this case, you can pass ignore_index=True:

```
In [107]: pd.concat([df1, df2], ignore_index=True)
Out[107]:
```

	a	b	c	d
0	1.246435	1.007189	-1.296221	0.274992
1	0.228913	1.352917	0.886429	-2.001637
2	-0.371843	1.669025	-0.438570	-0.539741
3	-1.021228	0.476985	NaN	3.248944
4	0.302614	-0.577087	NaN	0.124121



데이터 합치기

Combining and Merging Datasets

Table 8-3. *concat* function arguments

Argument	Description
<code>objs</code>	List or dict of pandas objects to be concatenated; this is the only required argument
<code>axis</code>	Axis to concatenate along; defaults to 0 (along rows)
<code>join</code>	Either 'inner' or 'outer' ('outer' by default); whether to intersection (inner) or union (outer) together indexes along the other axes
<code>join_axes</code>	Specific indexes to use for the other $n-1$ axes instead of performing union/intersection logic
<code>keys</code>	Values to associate with objects being concatenated, forming a hierarchical index along the concatenation axis; can either be a list or array of arbitrary values, an array of tuples, or a list of arrays (if multiple-level arrays passed in <code>levels</code>)
<code>levels</code>	Specific indexes to use as hierarchical index level or levels if keys passed
<code>names</code>	Names for created hierarchical levels if keys and/or levels passed
<code>verify_integrity</code>	Check new axis in concatenated object for duplicates and raise exception if so; by default (<code>False</code>) allows duplicates
<code>ignore_index</code>	Do not preserve indexes along concatenation axis, instead producing a new <code>range(total_length)</code> index



Combining Data with Overlap

There is another data combination situation that can't be expressed as either a merge or concatenation operation. You may have two datasets whose indexes overlap in full or part. As a motivating example, consider NumPy's `where` function, which performs the array-oriented equivalent of an if-else expression:

```
In [108]: a = pd.Series([np.nan, 2.5, np.nan, 3.5, 4.5, np.nan],
.....:                  index=['f', 'e', 'd', 'c', 'b', 'a'])
```

```
In [109]: b = pd.Series(np.arange(len(a), dtype=np.float64),
.....:                  index=['f', 'e', 'd', 'c', 'b', 'a'])
```

```
In [110]: b[-1] = np.nan
```

```
In [111]: a
Out[111]:
f    NaN
e    2.5
d    NaN
c    3.5
b    4.5
a    NaN
dtype: float64
```

```
In [112]: b
Out[112]:
f    0.0
e    1.0
d    2.0
c    3.0
b    4.0
a    NaN
dtype: float64
```

```
In [113]: np.where(pd.isnull(a), b, a)
Out[113]: array([ 0. ,  2.5,  2. ,  3.5,  4.5, nan])
```

Series has a `combine_first` method, which performs the equivalent of this operation along with pandas's usual data alignment logic:

```
In [114]: b[:-2].combine_first(a[2:])
Out[114]:
a    NaN
b    4.5
c    3.0
d    2.0
e    1.0
f    0.0
dtype: float64
```

With DataFrames, `combine_first` does the same thing column by column, so you can think of it as “patching” missing data in the calling object with data from the object you pass:

```
In [115]: df1 = pd.DataFrame({'a': [1., np.nan, 5., np.nan],
.....:                      'b': [np.nan, 2., np.nan, 6.],
.....:                      'c': range(2, 18, 4)})

In [116]: df2 = pd.DataFrame({'a': [5., 4., np.nan, 3., 7.],
.....:                      'b': [np.nan, 3., 4., 6., 8.]})
```

```
In [117]: df1
Out[117]:
   a  b  c
0  1.0 NaN 2
1 NaN 2.0 6
2  5.0 NaN 10
3 NaN 6.0 14
```

```
In [118]: df2
Out[118]:
   a  b
0  5.0 NaN
1  4.0 3.0
2 NaN 4.0
3  3.0 6.0
4  7.0 8.0
```

```
In [119]: df1.combine_first(df2)
Out[119]:
   a  b  c
0  1.0 NaN 2.0
1  4.0 2.0 6.0
2  5.0 4.0 10.0
3  3.0 6.0 14.0
4  7.0 8.0 NaN
```



8.3 Reshaping and Pivoting

There are a number of basic operations for rearranging tabular data. These are alternately referred to as *reshape* or *pivot* operations.

Reshaping with Hierarchical Indexing

Hierarchical indexing provides a consistent way to rearrange data in a DataFrame. There are two primary actions:

`stack`

This “rotates” or pivots from the columns in the data to the rows

`unstack`

This pivots from the rows into the columns

I’ll illustrate these operations through a series of examples. Consider a small DataFrame with string arrays as row and column indexes:

```
In [120]: data = pd.DataFrame(np.arange(6).reshape((2, 3)),
.....:                        index=pd.Index(['Ohio', 'Colorado'], name='state'),
.....:                        columns=pd.Index(['one', 'two', 'three'],
.....:                                       name='number'))
```

```
In [121]: data
Out[121]:
number  one  two  three
state
Ohio      0   1   2
Colorado  3   4   5
```

Using the `stack` method on this data pivots the columns into the rows, producing a Series:

```
In [122]: result = data.stack()

In [123]: result
Out[123]:
state  number
Ohio   one      0
       two      1
       three     2
Colorado one     3
        two     4
        three    5
dtype: int64
```

From a hierarchically indexed Series, you can rearrange the data back into a DataFrame with `unstack`:

```
In [124]: result.unstack()
Out[124]:
number  one  two  three
state
Ohio      0   1   2
Colorado  3   4   5
```




재형성과 피벗

Reshaping and Pivoting

By default the innermost level is unstacked (same with `stack`). You can unstack a different level by passing a level number or name:

```
In [125]: result.unstack(0)
Out[125]:
state  Ohio  Colorado
number
one      0         3
two      1         4
three    2         5

In [126]: result.unstack('state')
Out[126]:
state  Ohio  Colorado
number
one      0         3
two      1         4
three    2         5
```

Unstacking might introduce missing data if all of the values in the level aren't found in each of the subgroups:

```
In [127]: s1 = pd.Series([0, 1, 2, 3], index=['a', 'b', 'c', 'd'])

In [128]: s2 = pd.Series([4, 5, 6], index=['c', 'd', 'e'])

In [129]: data2 = pd.concat([s1, s2], keys=['one', 'two'])

In [130]: data2
Out[130]:
one  a    0
     b    1
     c    2
     d    3
two  c    4
     d    5
     e    6
dtype: int64

In [131]: data2.unstack()
Out[131]:
      a    b    c    d    e
one  0.0  1.0  2.0  3.0  NaN
two  NaN  NaN  4.0  5.0  6.0
```



재형성과 피벗

Reshaping and Pivoting

Stacking filters out missing data by default, so the operation is more easily invertible:

```
In [132]: data2.unstack()
Out[132]:
```

	a	b	c	d	e
one	0.0	1.0	2.0	3.0	NaN
two	NaN	NaN	4.0	5.0	6.0

```
In [133]: data2.unstack().stack()
Out[133]:
```

one	a	0.0
	b	1.0
	c	2.0
	d	3.0
two	c	4.0
	d	5.0
	e	6.0

```
dtype: float64

In [134]: data2.unstack().stack(dropna=False)
Out[134]:
```

one	a	0.0
	b	1.0
	c	2.0
	d	3.0
	e	NaN
two	a	NaN
	b	NaN
	c	4.0
	d	5.0
	e	6.0

```
dtype: float64
```

When you unstack in a DataFrame, the level unstacked becomes the lowest level in the result:

```
In [135]: df = pd.DataFrame({'left': result, 'right': result + 5},
.....:                      columns=pd.Index(['left', 'right'], name='side'))

In [136]: df
Out[136]:
```

side		left	right
state	number		
	Ohio	one	0
		two	1
Colorado	one	3	8
	two	4	9
	three	5	10

```
In [137]: df.unstack('state')
Out[137]:
```

side	left	right
state	Ohio	Colorado
number		
one	0	3
two	1	4
three	2	5

When calling stack, we can indicate the name of the axis to stack:

```
In [138]: df.unstack('state').stack('side')
Out[138]:
```

state		Colorado	Ohio
number	side		
	one	left	3
		right	8
two	left	4	1
	right	9	6
three	left	5	2
	right	10	7



Pivoting “Long” to “Wide” Format

A common way to store multiple time series in databases and CSV is in so-called *long* or *stacked* format. Let’s load some example data and do a small amount of time series wrangling and other data cleaning:

```
In [139]: data = pd.read_csv('examples/macrodta.csv')

In [140]: data.head()
Out[140]:
```

	year	quarter	realgdp	realcons	realinv	realgovt	realdpi	cpi	\
0	1959.0	1.0	2710.349	1707.4	286.898	470.045	1886.9	28.98	
1	1959.0	2.0	2778.801	1733.7	310.859	481.301	1919.7	29.15	
2	1959.0	3.0	2775.488	1751.8	289.226	491.260	1916.4	29.35	
3	1959.0	4.0	2785.204	1753.7	299.356	484.052	1931.3	29.37	
4	1960.0	1.0	2847.699	1770.5	331.722	462.199	1955.5	29.54	

	m1	tbilrate	unemp	pop	infl	realint
0	139.7	2.82	5.8	177.146	0.00	0.00
1	141.7	3.08	5.1	177.830	2.34	0.74
2	140.5	3.82	5.3	178.657	2.74	1.09
3	140.0	4.33	5.6	179.386	0.27	4.06
4	139.6	3.50	5.2	180.007	2.31	1.19


```
In [141]: periods = pd.PeriodIndex(year=data.year, quarter=data.quarter,
.....:                             name='date')

In [142]: columns = pd.Index(['realgdp', 'infl', 'unemp'], name='item')

In [143]: data = data.reindex(columns=columns)

In [144]: data.index = periods.to_timestamp('D', 'end')

In [145]: ldata = data.stack().reset_index().rename(columns={0: 'value'})
```

We will look at `PeriodIndex` a bit more closely in [Chapter 11](#). In short, it combines the year and quarter columns to create a kind of time interval type.

Now, `ldata` looks like:

```
In [146]: ldata[:10]
Out[146]:
```

	date	item	value
0	1959-03-31	realgdp	2710.349
1	1959-03-31	infl	0.000
2	1959-03-31	unemp	5.800
3	1959-06-30	realgdp	2778.801
4	1959-06-30	infl	2.340
5	1959-06-30	unemp	5.100
6	1959-09-30	realgdp	2775.488
7	1959-09-30	infl	2.740
8	1959-09-30	unemp	5.300
9	1959-12-31	realgdp	2785.204

This is the so-called *long* format for multiple time series, or other observational data with two or more keys (here, our keys are date and item). Each row in the table represents a single observation.



재형성과 피벗

Reshaping and Pivoting

Data is frequently stored this way in relational databases like MySQL, as a fixed schema (column names and data types) allows the number of distinct values in the `item` column to change as data is added to the table. In the previous example, `date` and `item` would usually be the primary keys (in relational database parlance), offering both relational integrity and easier joins. In some cases, the data may be more difficult to work with in this format; you might prefer to have a DataFrame containing one column per distinct `item` value indexed by timestamps in the `date` column. DataFrame's `pivot` method performs exactly this transformation:

```
In [147]: pivoted = ldata.pivot('date', 'item', 'value')
```

```
In [148]: pivoted
```

```
Out[148]:
```

item	infl	realgdp	unemp
date			
1959-03-31	0.00	2710.349	5.8
1959-06-30	2.34	2778.801	5.1
1959-09-30	2.74	2775.488	5.3
1959-12-31	0.27	2785.204	5.6
1960-03-31	2.31	2847.699	5.2
1960-06-30	0.14	2834.390	5.2
1960-09-30	2.70	2839.022	5.6
1960-12-31	1.21	2802.616	6.3
1961-03-31	-0.40	2819.264	6.8
1961-06-30	1.47	2872.005	7.0
...	...	...	...
2007-06-30	2.75	13203.977	4.5
2007-09-30	3.45	13321.109	4.7
2007-12-31	6.38	13391.249	4.8

The first two values passed are the columns to be used respectively as the row and column index, then finally an optional value column to fill the DataFrame. Suppose you had two value columns that you wanted to reshape simultaneously:

```
In [149]: ldata['value2'] = np.random.randn(len(ldata))
```

```
In [150]: ldata[:10]
```

```
Out[150]:
```

	date	item	value	value2
0	1959-03-31	realgdp	2710.349	0.523772
1	1959-03-31	infl	0.000	0.000940
2	1959-03-31	unemp	5.800	1.343810
3	1959-06-30	realgdp	2778.801	-0.713544
4	1959-06-30	infl	2.340	-0.831154
5	1959-06-30	unemp	5.100	-2.370232
6	1959-09-30	realgdp	2775.488	-1.860761
7	1959-09-30	infl	2.740	-0.860757
8	1959-09-30	unemp	5.300	0.560145
9	1959-12-31	realgdp	2785.204	-1.265934



재형성과 피벗

Reshaping and Pivoting

By omitting the last argument, you obtain a DataFrame with hierarchical columns:

```
In [151]: pivoted = ldata.pivot('date', 'item')
```

```
In [152]: pivoted[:5]
```

```
Out[152]:
```

	value			value2		
item	infl	realgdp	unemp	infl	realgdp	unemp
date						
1959-03-31	0.00	2710.349	5.8	0.000940	0.523772	1.343810
1959-06-30	2.34	2778.801	5.1	-0.831154	-0.713544	-2.370232
1959-09-30	2.74	2775.488	5.3	-0.860757	-1.860761	0.560145
1959-12-31	0.27	2785.204	5.6	0.119827	-1.265934	-1.063512
1960-03-31	2.31	2847.699	5.2	-2.359419	0.332883	-0.199543

```
In [153]: pivoted['value'][:5]
```

```
Out[153]:
```

item	infl	realgdp	unemp
date			
1959-03-31	0.00	2710.349	5.8
1959-06-30	2.34	2778.801	5.1
1959-09-30	2.74	2775.488	5.3
1959-12-31	0.27	2785.204	5.6
1960-03-31	2.31	2847.699	5.2

Note that pivot is equivalent to creating a hierarchical index using set_index followed by a call to unstack:

```
In [154]: unstacked = ldata.set_index(['date', 'item']).unstack('item')
```

```
In [155]: unstacked[:7]
```

```
Out[155]:
```

	value			value2		
item	infl	realgdp	unemp	infl	realgdp	unemp
date						
1959-03-31	0.00	2710.349	5.8	0.000940	0.523772	1.343810
1959-06-30	2.34	2778.801	5.1	-0.831154	-0.713544	-2.370232
1959-09-30	2.74	2775.488	5.3	-0.860757	-1.860761	0.560145
1959-12-31	0.27	2785.204	5.6	0.119827	-1.265934	-1.063512
1960-03-31	2.31	2847.699	5.2	-2.359419	0.332883	-0.199543
1960-06-30	0.14	2834.390	5.2	-0.970736	-1.541996	-1.307030
1960-09-30	2.70	2839.022	5.6	0.377984	0.286350	-0.753887



Pivoting “Wide” to “Long” Format

An inverse operation to pivot for DataFrames is `pandas.melt`. Rather than transforming one column into many in a new DataFrame, it merges multiple columns into one, producing a DataFrame that is longer than the input. Let's look at an example:

```
In [157]: df = pd.DataFrame({'key': ['foo', 'bar', 'baz'],
.....:                      'A': [1, 2, 3],
.....:                      'B': [4, 5, 6],
.....:                      'C': [7, 8, 9]})
```

```
In [158]: df
Out[158]:
   A  B  C key
0  1  4  7 foo
1  2  5  8 bar
2  3  6  9 baz
```

The 'key' column may be a group indicator, and the other columns are data values. When using `pandas.melt`, we must indicate which columns (if any) are group indicators. Let's use 'key' as the only group indicator here:

```
In [159]: melted = pd.melt(df, ['key'])
```

```
In [160]: melted
Out[160]:
   key variable  value
0  foo         A       1
1  bar         A       2
2  baz         A       3
3  foo         B       4
4  bar         B       5
5  baz         B       6
6  foo         C       7
7  bar         C       8
8  baz         C       9
```

Using `pivot`, we can reshape back to the original layout:

```
In [161]: reshaped = melted.pivot('key', 'variable', 'value')
```

```
In [162]: reshaped
Out[162]:
variable A  B  C
key
bar      2  5  8
baz      3  6  9
foo      1  4  7
```

Since the result of `pivot` creates an index from the column used as the row labels, we may want to use `reset_index` to move the data back into a column:

```
In [163]: reshaped.reset_index()
Out[163]:
variable key  A  B  C
0      bar   2  5  8
1      baz   3  6  9
2      foo   1  4  7
```

You can also specify a subset of columns to use as value columns:

```
In [164]: pd.melt(df, id_vars=['key'], value_vars=['A', 'B'])
Out[164]:
   key variable  value
0  foo         A       1
1  bar         A       2
2  baz         A       3
3  foo         B       4
4  bar         B       5
5  baz         B       6
```



재형성과 피벗

Reshaping and Pivoting

`pandas.melt` can be used without any group identifiers, too:

```
In [165]: pd.melt(df, value_vars=['A', 'B', 'C'])
```

```
Out[165]:
```

	variable	value
0	A	1
1	A	2
2	A	3
3	B	4
4	B	5
5	B	6
6	C	7
7	C	8
8	C	9

```
In [166]: pd.melt(df, value_vars=['key', 'A', 'B'])
```

```
Out[166]:
```

	variable	value
0	key	foo
1	key	bar
2	key	baz
3	A	1
4	A	2
5	A	3
6	B	4
7	B	5
8	B	6

Q&A

질의응답 (5분)



끝. 감사합니다.

수업 듣느라 수고하셨습니다.

